

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

Music Managment Application

Bachelor's Thesis

KIRILL VOROZHTSOV

Brno, Spring 2026

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

Music Managment Application

Bachelor's Thesis

KIRILL VOROZHTSOV

Advisor: Mgr. Luděk Bártek, Ph.D

Brno, Spring 2026



Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

During the preparation of this thesis, the following AI tools were used:

- **ChatGPT:** Used for writing most of the tests, debugging and refactoring, as well as LaTeX formatting.
- **Grammarly:** Used for text corrections.

Kirill Vorozhtsov

Advisor: Mgr. Luděk Bártek, Ph.D

Acknowledgements

I would like to thank Mgr. Luděk Bártek, Ph.D., for his time and the valuable advice he provided for this thesis.

Abstract

This bachelor's thesis aims to develop a web-based music management and streaming platform focused on social interactions between users.

The theoretical part presents a survey that analyzes people's music listening habits, showing that such a platform could attract a user base. It then compares existing music-related services with built-in social features and derives functional and non-functional requirements for the proposed system. Then, based on these requirements, suitable technologies and architectural patterns are selected.

The practical part describes the built application. It explains the choices behind each feature, focusing on what is not clear from the diagrams and the source code. It then discusses the user testing outcomes and lists possible improvements.

The result is a fully functional and deployed application.

Keywords

Audio Streaming, Python, JavaScript, Web Application, WebSockets

Contents

1	Introduction	1
2	Music Consumption Survey	3
2.1	Methods	3
2.2	Results	3
2.2.1	Engagement	3
2.2.2	Listening Methods	4
2.2.3	Social Interactions	4
2.3	Summary	6
3	Existing Platforms	7
3.1	Streaming Services	7
3.1.1	Spotify	7
3.1.2	VK Music	8
3.1.3	SoundCloud	8
3.1.4	Bandcamp	8
3.2	Forums, Blogs, and Other Music Media	8
3.2.1	Rate Your Music	9
3.2.2	last.fm	9
3.3	General-Purpose Platforms	9
4	Specification	12
4.1	Important Terms	12
4.2	Use Cases	14
4.3	Functional Requirements	14
4.4	Non-Functional Requirements	16
5	Implementation Planning	17
5.1	Audio Playback	17
5.2	Authentication	19
5.3	Server-Client Communication	21
5.4	Backend	24
5.4.1	Language	24
5.4.2	Python Framework Comparison	24
5.4.3	Database	26
5.5	Frontend	27

5.5.1	Application Type	27
5.5.2	Language	28
5.5.3	Framework	28
5.6	Summary	29
6	Implementation	31
6.1	Stack	31
6.2	Audio Processing and Streaming	32
6.2.1	Streaming Protocols	32
6.2.2	Packaging	32
6.2.3	Transcoding	32
6.2.4	Audio Preprocessing	33
6.2.5	Frontend Playback	35
6.3	WebSocket Communication	35
6.3.1	Backend Handling	35
6.3.2	Frontend Handling	36
6.4	User Management	37
6.4.1	User Data Model	37
6.4.2	Backend	38
6.4.3	Frontend	38
6.5	Account Creation and Authentication	38
6.5.1	Backend	39
6.5.2	Frontend	39
6.6	Songs and Collections	40
6.6.1	Data Model	40
6.6.2	Backend	41
6.6.3	Frontend	41
6.7	Audio Upload/Album Creation	41
6.8	Song Queue	43
6.8.1	Data Structure	43
6.8.2	Queue and PlaybackContext	43
6.8.3	QueueSource and Lazy Filling	44
6.9	Playback and Device Handling	44
6.9.1	Shared State and Version Counter	45
6.9.2	Position Estimation on Passive Tabs	46
6.9.3	Device Activation	46
6.9.4	Browser Autoplay Handling	47
6.9.5	Frontend Playback Control	47

6.10	Publications	48
6.11	Search	49
6.12	Notifications	50
6.13	Security	51
6.13.1	Transport and Origin	51
6.13.2	Authentication and Session Management	51
6.13.3	XSS and CSRF	52
6.13.4	Input Validation	52
6.13.5	WebSocket Security	52
6.13.6	HTTP Response Headers	52
6.14	Deployment	53
7	User Testing and Further Improvements	54
7.1	Further Improvements	54
7.1.1	Out-of-Scope Items	54
7.1.2	Search and Recommendations	54
7.1.3	Playback Synchronization	54
7.1.4	Backend Refactors	55
7.1.5	Frontend Cache Strategy	55
7.1.6	Test Coverage	55
8	Conclusion	56
8.1	Closing Remarks	56
A	Audio Processing Pipeline Details	58
A.1	Streaming Protocols	58
A.2	Packaging	58
A.3	Processing Tools	58
A.4	Transcoding	59
A.4.1	AAC Encoder Choice	59
A.4.2	Bitrate Strategy: CBR Versus VBR	59
A.4.3	Generation Loss	60
A.5	Audio Preprocessing	60
A.5.1	Input Validation Limits	60
A.5.2	FFprobe Invocation	61
A.5.3	Channel Normalization	62
A.5.4	Sample Rate Normalization	62
A.5.5	Bit Depth Conversion	62

A.5.6	Loudness Theory	63
A.5.7	Why Loudness Is Measured After Transcoding .	63
A.6	Pipeline Invocations	64
A.6.1	FFmpeg Transcoding Flags	64
A.6.2	Shaka Packager Flags	65
A.6.3	ebur128 Loudness Measurement Flags	66
Bibliography		68

List of Tables

3.1	Comparison of Service-Specific Social Features	11
-----	--	----

List of Figures

1 Introduction

Music plays an important role in everyday life. Rentfrow [1] shows that music can affect mood, change how people see themselves, and help them bond with others. Moreover, music is also a social activity. For example, people listen to it together at clubs, festivals, and similar events. [2] In addition, over the past ten years, music streaming has grown a lot. As a result, it is now the main way most people listen to music. [3]

Many music platforms offer social features, but each one supports only a subset, as shown in **Chapter 3**. No single service brings these features together. A survey done for this thesis (**Chapter 2**) shows that there is demand for such a combination, with most respondents saying they would use more social features on a music platform.

To address this gap, this bachelor's thesis aims to develop a web-based music management and streaming platform focused on social interactions between users. The platform combines a personal music library, audio Streaming, and social features such as Chats, Comments, and Feeds.

The deployed application is available at <https://musikk.stream>. This work is divided into the following chapters:

- **Chapter 2** Presents the outcomes of a survey illustrating how people consume music, how prevalent it is in social interactions, and why this type of platform can be relevant.
- **Chapter 3** Explores which social features existing streaming platforms and other music-related services already offer.
- **Chapter 4** Defines the functional and non-functional requirements for the application.
- **Chapter 5** Describes the choices of technologies and architectural patterns used by the application.
- **Chapter 6** Describes the development process and implementation details.
- **Chapter 7** Reports on user testing of the deployed application and lists known gaps and further improvements.

- **Chapter 8** Summarises the outcome of the work.

Conventions used in this thesis:

- **Figures**

Most of the diagrams and survey charts in this thesis are too large to embed inline. They are referenced by their file path in the project repository, for example `text/charts/age.png` or `text/diagrams/HighLevelSystem.png`. The complete sets are kept under `text/diagrams/` and `text/charts/`.

- **Citations**

Sources are cited on the first mention of a work, term, or claim. Later mentions of the same work or term are not re-cited.

2 Music Consumption Survey

To support the choice of topic and show that a music streaming platform focused on social interactions could be useful, a short survey was conducted. It looked at personal music listening and discovery habits, how people use platforms, how people interact about music, and opinions on possible social features on the platform.

2.1 Methods

The service chosen for the questionnaire was Google Forms [4]. It has a simple interface for creating surveys, makes the form easy to share, and can export results as CSV files.

The questionnaire consists of 15 questions. Most are multiple-choice with fixed options, and some allow a custom answer. All custom answers are grouped under the “other” category in the figures below. Answers in their original form can be found in the full survey.

The complete questionnaire is included in the thesis repository under `text/extras/survey-questions.pdf` and online.¹

2.2 Results

This section presents the survey results and discusses what they mean. Not all questions are discussed. For example, some questions about music discovery are left out.

A total of 119 people took part, with most being in the 18 to 30 age group, with the exact distribution shown in `text/charts/age.png`.

2.2.1 Engagement

First, the survey needed to find out whether respondents listen to music often and whether they use streaming platforms for it.

Over 50% of respondents reported listening to more than 500 songs per month, i.e., 16 songs a day on average (`text/charts/`

1. https://docs.google.com/forms/d/1vhhAu_SfuHV4xy6JoDaRsM5uCE1Yn_csTmN8u4ZlpHc

songamountmonth.png). Nearly 80% stated that they listen to music daily (text/charts/listeningregularity.png).

The numbers show that people are very interested in music, and the next step was to find out how they mostly access it.

2.2.2 Listening Methods

As shown in text/charts/consumption_method.png, the share of respondents using streaming platforms across all age groups is close to 100%, with slightly higher numbers in the younger age groups. Although downloaded files and physical media are still used, especially among people aged 30 to 60, they are usually only a secondary option next to streaming services.

The question about the popularity of individual platforms showed that Spotify is the most used one, which matches global statistics [5]. However, Yandex Music, coming in second place, needs further explanation. Most of the respondents were from Russian-speaking countries, specifically Russia, and, with many Western companies leaving the Russian market in 2022, including music streaming services [6], most users have switched to local products: Yandex Music [7], VK Music [8], and Zvuk [9]. This also explains why other popular region-specific platforms, such as Amazon Music (US), QQ Music (China), and JioSaavn (India), do not appear in the survey.

See text/charts/streamingplatform.png for the complete statistics.

The survey showed that streaming services are indeed widely used. The next step was to look at how often music-related social interactions happen.

2.2.3 Social Interactions

The results of Question 10 show that nearly all respondents (99.2%) said they listen to music with others at least sometimes, mostly in offline settings such as gatherings or car journeys (text/charts/listentogethermethod.png). Online shared listening methods, although less common, were also mentioned by a quarter of respondents.

Question 11 ([text/charts/listentogetherfreq.png](#)) looks at how often these interactions happen: the results show that many respondents take part in shared listening regularly. Around 60% said they do so at least a few times per month, with a smaller group reporting almost daily interactions. This supports the earlier idea in **Chapter 1** that listening to music is a common social activity.

The next question asks: “How often do you/your friends share/discuss music?”. While the previous question focused on real-time in-person shared listening, this one looks at more deliberate and active music-related interactions, such as sharing songs, recommending music, or discussing it. Here, music is the main point of the activity, not background. The same trend as before appears in the responses: over 70% ([text/charts/sharefreq.png](#)) said that they share music at least several times a month, with many doing so weekly or more often. This further confirms that music plays an active social role.

Question 13, “How does that happen?” (in terms of music sharing), asks about the specific ways people share music. Many respondents ([text/charts/sharemethod.png](#)) said that they usually send links from streaming platforms through external messaging apps. While this shows a clear demand for sharing music online, it also shows a gap among platforms, as these interactions stay split across apps. Hence, allowing this sharing directly within the streaming app could give a smoother and more consistent experience.

Responses to the question “Would you say that you know a lot of people with similar music taste?” were mixed, with a near-even split. This suggests that while some users already have a social circle with similar music taste, many do not. In addition, when asked “Would you like to connect with others who share your musical taste, or influence those around you to explore and appreciate the music you enjoy?”, most answered yes.

This data further supports the idea that social discovery features could be useful in a streaming platform. Moreover, over 60% of participants said in Question 15 that they would use extra social features if available on a streaming platform.

2.3 Summary

Overall, the results show that people listen to music often, with streaming services being their main way of accessing it. The data also show that music plays an important role in everyday life and often appears in social situations.

Yet, existing platforms only partly support interaction between users. This makes the idea of a music streaming service with built-in social features both relevant and likely to meet real user needs.

3 Existing Platforms

In this chapter, existing platforms and services are looked at to get a better understanding of the tools people use when interacting with music. Additionally, this chapter gives an idea of the possible features that could be added to the final application.

First, streaming services are discussed, as the main goal of the thesis is to build a music streaming system. Then, other music-related services (e.g., forums) are shown, as they offer other ways of engaging with audio content. Some of those ideas could be useful for the future platform. Lastly, platforms such as YouTube and TikTok are looked at, because the survey has shown that they are a popular way to interact with music (`text/charts/discoverymethods.png`).

3.1 Streaming Services

As already shown in `text/charts/consumption_method.png`, and further confirmed by the study of the International Federation of the Phonographic Industry [10], the most common way of listening to and discovering music is through streaming services. Although many options are available, this section only looks at those that are both popular and have unique features. The descriptions, rather than giving a general description of each platform, will focus on the service-specific social features.

3.1.1 Spotify

One of the most notable social features on Spotify is the “Spotify Jam” feature [11]. It lets people create a shared song queue that is then synced among all connected users. Moreover, volume, the order of songs, and other playback aspects can be controlled individually by each user.

Another notable feature is “Friend Activity” [12], which shows what the followers of a given user are currently listening to.

Another feature is “Listening Parties”: these are live chats with a limited capacity that can be joined for a short time when new music is

being released [13, 14]. This brings the offline shared music discovery experience online.

During the implementation of this thesis (the work has started in spring of 2025), Spotify has also added support for user-to-user chats (from September 2025) [15]. This further supports the claim that social features are wanted on music platforms.

3.1.2 VK Music

VK Music is a music service built into the VK social network. As a result, it is possible to send songs and playlists through private messages or add them to posts in groups. VK Music also has an “Updates” tab, which is filled with the audio content that any followed user or group has added to their “Liked Feed” [8].

3.1.3 SoundCloud

SoundCloud is one of the few platforms that lets its users leave comments and reactions on songs and playlists [16, 17]. In addition, each user has a feed made of personal uploads and reposts of others’ content [18], which is visible when visiting their profile.

3.1.4 Bandcamp

Every Bandcamp user has a profile with four tabs: “collection”, “wishlist”, “followers”, and “following”. [19] A useful feature is under the “following” tab: users can subscribe to available genres and then choose the ones they want to show to others viewing their profile. The “wishlist” tab is also useful, as it lets the users show what they are looking forward to listening to in the future.

This concludes the streaming services section. Services that are focused on music, but not on the streaming itself, are discussed next.

3.2 Forums, Blogs, and Other Music Media

Different resources are gathered under these broad terms, from professional music review websites to amateur and personal pages. Again, only widely used services that offer something interesting are listed.

3.2.1 Rate Your Music

“Rate Your Music is one of the largest music databases online. It is an incredible tool that can help you find and learn about new music to listen to.” [20]

This website is one of the largest platforms with user-created content. Every member can write reviews and rate music releases, contribute to the “wiki” made of genres, themed lists, and charts, and connect with other members of the forum.

3.2.2 last.fm

Last.fm tracks listening statistics and helps with music discovery. However, it also has a couple of notable social features [21]:

1. Global and personal events pages. In the former, users can browse the upcoming events near their location, and in the latter they can mark which events they plan to attend. Other users can then see, either on the event page or on that user’s profile, which events the user is interested in and/or plans to attend.
2. A “Neighbours” tab. The “Neighbours” tab shows other users with similar listening habits, based on shared artists and an overall compatibility score.
3. “Shouts” section. The “Shouts” section is a feed on each user’s profile, where the users can leave comments and reply to what others have written.

This section has shown a couple of possible features not previously seen in the listed streaming services, such as discussion forums and profile feeds.

The following section looks at platforms that are not mainly focused on music but still affect how people interact with it.

3.3 General-Purpose Platforms

As mentioned earlier, one of the most interesting results of the survey is the high number of people discovering music on platforms that are

not focused on music. Services like Instagram, YouTube, and TikTok are mostly used for sharing videos and other user-generated content. They have become closely tied to how people find music and listen to it. In recent years, these platforms have started to promote music more directly by letting users add tracks to their own content.

Instagram, for example, added a special audio tool that lets users include music in their short videos (“Reels” [22]), helping songs reach more people through visual content.

YouTube automatically detects songs used in videos and shows them in the description, making it easier to find and listen to them. [23]

TikTok has had an even bigger impact: many songs have gone viral and later became hits in charts and streaming apps because people use them in popular videos (these are often called “TikTok Hits”). [24]

This shows that big platforms are not just reacting to users’ interest in music, they actively help promote it by connecting songs with content that people want to watch and share.

An integration with such services would help the platform. A possible feature could be a badge stating “This track was in a TikTok that user Alice123 sent you today!”.

Summary

The table 3.1 summarizes the comparison between the features mentioned in the sections above.

Difficulty shows the possible complexity of implementation.

Mode assumes the typical context in which the feature is used.

Social Interaction Level shows user involvement when using the feature.

Overall, many music-related platforms with social features built in are available across the Internet. However, services that focus on the audio delivery itself often support only some of the possible features. This, in turn, forces users to use multiple platforms to meet their needs. This project aims to close that gap, gathering multiple social features in one app.

Based on the research done in the previous and current chapters, the following social features were chosen for implementation, as they

Table 3.1: Comparison of Service-Specific Social Features

Feature	Interaction Level	Mode	Difficulty
Chat	Very High	Pair/Group	High
Common Listening	High	Group	Very High
Forums/Wiki	High	Group	Medium
Personal Feed	High	Individual	Low
Song/Playlist Comments	High	Group	Low
Listening Parties	Medium	Group	Very High
Friends' Listening Activity	Medium	Pair	Low
Following Genres Display	Low	Individual	Medium
Real-Life Events Section	Low	Group	High
Music Wishlist	Low	Individual	Low
External Services Integration	Low	Pair/Group	High

offer a good balance between the complexity of implementation and the level of social interaction:

- Chat.
- Personal/Global Feed.¹
- Song/Playlist Comments.
- Friends' Listening Activity.

These and further requirements for the application are listed in **Chapter 4**.

1. In comparison to `last.fm`, a common view with posts from multiple authors is also provided.

4 Specification

In this chapter the general outline of the application is given via use cases and functional/non-functional requirements. The individual requirements are constructed based on the “MoSCoW” criteria [25]. “Must Have” requirements specify functionality without which the application cannot be considered finished. Those are the main features and technical aspects that must be implemented. “Should Have” requirements mostly represent features that would greatly improve the user experience, but are not strictly needed for the application to be fully working. “Could Have” requirements are reserved for possible future ideas which most probably would not be implemented in the current scope of the thesis.

4.1 Important Terms

Below is the list of important terms that appear throughout the requirements sections and further chapters:

- **Anonymous User:** A User who is not yet registered in the system or has not logged in.
- **User:** An object containing information such as email, display name and other User parameters.
- **User Profile:** A Frontend-only abstraction which shows Base User information.
- **Artist:** A User with additional privileges, for example, Song or Collection creation.
- **Follower/Followee:** A relationship between a User and another User where one subscribes to another.
- **Friend:** A Followee and Follower who are mutually subscribed to each other.
- **Song:** An object representing a processed audio file with additional metadata such as title or creation date.

- **Collection:** A container that holds multiple Songs.
- **Playlist:** A Collection consisting of pre-existing (already uploaded) Songs.
- **Album:** A Collection consisting of newly uploaded Songs.
- **Playback:** A process during which the User is receiving or manipulating audio data.
- **Playback Item:** A Song or a Collection.
- **Playback Queue:** A User-specific list of Playback Items which determines the playing Song order.
- **Liked Songs:** A private, system-managed Collection that holds the Songs a User has marked as liked.
- **History:** A private, system-managed Collection that records the Songs a User has played.
- **Playback Device:** A device on which the app is currently used.
- **Active Playback Device:** The User's Playback Device that is currently producing audio. All other Playback Devices are passive and mirror the Active Device's Playback position.
- **Friend Activity:** A display of the Songs currently listened to by the User's Friends.
- **Publication:** Textual User input.
- **Attachment:** Non-textual User input. For example, a Song attached to a Publication.
- **Reply:** The same as Publication, but created in relation to another, "parent" Publication.
- **Feed:** A list of Publications, either User-specific or Global (from multiple Users).
- **Chat:** A list of Publications specific to two or more Users.

4.2 Use Cases

While the requirements in the following sections list what the system must do, use cases show the same functionality from the actor's perspective. This can make further modeling simpler, since the minimal subset of actions any given User can take is defined explicitly.

The use cases are split into six groups: Account and Identity, Social, Library and Content, Playback, Search, and Communication. Each group has its own use case diagram. The Artist actor inherits from the User actor, so every use case available to a User is also available to an Artist.

The use case diagrams for each group are kept under `text/diagrams/`, for example `text/diagrams/UseCasesAccount.png`.

4.3 Functional Requirements

Must Have

- The system shall support User and Artist models.
- The system shall allow Anonymous Users to create a User model and log in to the system.
- The system shall allow Users to change model information (name, bio, avatar, etc.) after the creation.
- The system shall provide audio Playback over the network, with User-controlled Playback and a modifiable Playback Queue.
- The system shall let Users browse Playback Items, including a personal library of owned and liked content.
- The system shall support the creation of Playback Items. Playlists shall be created by all Users. Albums shall be created by Artists only.
- The system shall allow Users to mark individual Songs and entire Collections as liked.

- The system shall provide Chat capabilities, including direct Chats between two Users and group Chats with multiple members.
- The system shall support a Publication system, including Replies, Comments on public Collections, and per-User and global Feeds.
- The system shall allow Users to follow and unfollow other Users. Mutual follows shall be treated as a Friend relation.
- The system shall have a way to search Playback Items, Users, and Artists.
- The system shall display Friend Activity, that is, the Songs currently being played by the User's Friends.

Should Have

- The system shall support adding Attachments, such as Songs, Collections, or Users, to Publications and Chat messages.
- The system shall provide a Notification system.
- The system shall make it possible to filter content by the viewer's relation to the author.
- The system shall allow a User to switch the Active Playback Device.

Could Have

- The system shall make it possible for multiple Users to listen to Songs in a synchronized manner.
- The system shall provide an Artist interface that allows Artists to see statistics related to their content.
- The system shall have a forum section.
- The system shall provide content recommendations based on a User's listening history.

4.4 Non-Functional Requirements

Must Have

- The system shall provide access to its contents only to authorized Users. The only exceptions shall be the “Log In” and “Sign Up” pages.
- The system shall harden against common web threats (CSRF, XSS, etc.) and follow defense in depth, so that stored content stays protected and misconfigured access does not expose data.
- The system shall store and transport sensitive User information safely. I.e., it should use safe transport protocols, such as HTTPS and WSS.
- The system shall, in case of unexpected failure, remain rendered on the screen and show the appropriate non-technical message to the User.
- The system shall minimize the network load it has on the server and the User device. That includes reducing the data footprint and implementing eager loading strategies.

Should Have

- The system shall provide a responsive search method that reacts dynamically to User input.
- The system shall keep Playback position and the current Song synchronized across a User’s open Playback Devices.

Could Have

- The system shall support offline Playback.
- The system should integrate a Digital Rights Management (DRM) mechanism.

The next chapter presents the high-level technology choices used for the implementation.

5 Implementation Planning

Before the actual implementation could begin, it was necessary to choose the languages, frameworks, and other technologies that would decide the overall architecture of the application. Since the goal is a web application, this chapter only looks at web-suitable technologies. This chapter discusses the following areas: Audio Playback, Authentication, Server-Client Communication, Backend, and Frontend.

5.1 Audio Playback

Audio Playback is the most important aspect of the application and is researched first. There are four main approaches used today: Basic Download, Progressive Download, Streaming, and Peer-to-Peer.

- **Basic Download**

With Basic Download, the entire file must arrive before Playback can start. The client issues a standard HTTP GET, and the server returns the media as binary data in the response body. [26] For high-quality files this means large storage space and a long download time, and once a download starts the system cannot adapt to network changes.

- **Progressive Download**¹

Progressive Download lets Playback start before the full file arrives. The client fetches a short initial part over HTTP, begins Playback, and requests more parts as needed, usually via HTTP byte-range requests. [28] The player relies on audio container metadata (e.g., the header of an MP4 file) that maps Playback time to byte offsets, so it can request the next few seconds of audio or seek to a different position. [29]

Storage becomes less of an issue than with Basic Download, since the player only needs a small in-memory buffer and older data can be discarded once played. Adaptive Playback is also

1. Sometimes called “Pseudo-Streaming”. [27]

possible, but it is not standardized. It requires custom preprocessing of the audio on the server and custom client logic to choose between different quality versions during Playback.

- **Segmented HTTP Streaming**

Segmented HTTP Streaming builds adaptive Playback into the protocol itself, rather than leaving it to the application. The audio file is split into many small chunks, and a manifest is generated that lists the chunks and states the time range of each chunk (usually a couple of seconds). [30] The most common implementations of this approach are MPEG-DASH and HLS, both of which are described in **Chapter 6**. The client downloads individual chunks instead of working with byte ranges in a single large file, and the manifest provides the chunk-to-time mapping needed for Playback and seeking.

The original audio file can also be encoded into several versions of different size and quality, all referenced by the same manifest. The client picks chunks based on network conditions or device capabilities, and since each chunk is a separate HTTP resource, retries and caching are handled efficiently by standard HTTP caches and CDNs. [31]

- **Peer-to-Peer**

With Peer-to-Peer transfer, audio data is shared directly between Users' devices rather than served from a central server. [32] This can reduce server load, but Playback speed depends on peer availability and network conditions.

Streaming was chosen as the Playback method for this thesis.

Basic Download was excluded because it requires downloading the whole file before Playback can start, which makes the application less usable. Progressive Download improves startup time and seeking. However, adapting to network changes is not standardized and would require custom audio preprocessing on the server and custom client logic. Peer-to-Peer transfer was also excluded, since it would rely on Users' devices as sources of audio data, so Playback speed would depend on peer availability and location.

Segmented HTTP Streaming addresses these issues by defining adaptive Playback directly in the manifest and delivering the audio

as small chunk files. Additionally, its integration into the service is straightforward (see **Chapter 6**).

The next choice was the authentication method, since it affects both the Frontend and Backend design.

5.2 Authentication

There are many different methods of authentication available, but only those suitable for web applications are considered: Basic Authentication, Session-Based Authentication, Token Authentication, and Third-Party Authentication (OAuth2).

- **Basic Authentication**

With Basic Authentication, every request sends the User's actual password, Base64-encoded but not encrypted. The credentials are placed in the HTTP Authorization header, and the browser typically caches them after the first successful login and attaches the header to later requests on its own. [33]

If this value leaks, for instance into a log after HTTPS is terminated at an internal proxy [34], the password itself leaks rather than an opaque token. Revoking access therefore requires a password change. The protocol also has no logout step or per-device sessions, so signing a User out requires Frontend-side workarounds.

- **Cookie-Based Session Authentication**

Session authentication keeps the credential on the server and only sends an opaque identifier to the client. The User logs in with their credentials, the server creates a *session* record holding the relevant User information (ID, roles, and so on), and instructs the browser to set a cookie with the session ID. [35] This ID is sent on later requests instead of the credentials and identifies the logged-in User.

Compared to Basic Authentication, this allows explicit server-side logout (by deleting or invalidating sessions), as well as per-session policies such as Device tracking and inactivity timeouts. The main trade-off is the need for a session store, typically a

relational database or an in-memory key-value store, which adds infrastructure and a potential bottleneck if not configured correctly.

- **Token Authentication**

Token Authentication replaces raw credentials with a token issued by the server after login, which the client sends in the Authorization header under the Bearer identifier. For a single-page application, the main trade-off is where the token is stored in the browser: if held in `localStorage`, it is reachable from any script on the page, so a Cross-Site Scripting (XSS) injection can read it [36], while session cookies marked `HttpOnly` are not exposed to scripts. [37]

Tokens come in two common variants. A *database-backed token* is stored on the server alongside the User record and looked up on every request, similar to a session but sent in a header instead of a cookie. A *self-contained token*, such as a JSON Web Token (JWT) [38], holds the User identity and permissions in its payload and is verified by signature, with no lookup. Self-contained tokens simplify horizontal scaling, but revocation is hard. Once issued, a token stays valid until it expires, unless a revocation list is kept, which adds state and coordination. In both variants, the Frontend has to attach the token to every request and handle refresh explicitly, while cookies are sent by the browser automatically.

- **Third-Party Authentication (OAuth2)**

Third-Party Authentication complements one of the methods above, it does not replace them. It hands off login to an external identity provider via the OAuth 2.0 protocol, such as Google or GitHub. [39] The application redirects the User to the provider, receives an authorization code in the callback, and exchanges it for tokens that identify the User. Password storage, multi-factor authentication, and account recovery are then handed off to the provider. However, this cannot be the only authentication method an application offers, since it would require every User to have an account with one of the supported providers.

Based on these characteristics, Session Authentication was chosen.

Compared to Basic Authentication, it avoids sending User credentials with every request, supports explicit logout, per-session security policies, and extra session-related data. Compared to Token Authentication, sessions offer simpler revocation, do not require managing tokens, and are good enough for a web application that does not need to share authentication across services. Third-Party Authentication based on OAuth 2.0 can be added on top of this design.

The next topic is server-client communication.

5.3 Server-Client Communication

Some of the features outlined in **Chapter 4** (such as Playback handling and Chat) require bi-directional communication. Therefore, a way for the server to talk to the client is needed.

Because this is a web application, techniques such as Webhooks, gRPC, Pub/Sub, and others that are not directly supported by browsers are not discussed. Traditional Polling is also not considered because it performs poorly. [40] P2P techniques (e.g., WebRTC) are not listed because the application's architecture is client-server, not peer-to-peer.

This thesis focuses on four commonly used methods for server-client communication: Long Polling, Server-Sent Events, WebSockets, and Push API.

- **Long Polling**

Long Polling simulates server-to-client communication by holding an HTTP request open until the server has something to send, after which the client immediately issues another request. [41] It works in environments where server messages are rare, but when many small messages are sent often, it adds significant overhead from repeatedly opening and closing HTTP connections. [40]

- **Server-Sent Events (SSE)**

SSE is a one-way protocol that lets the server push data to the client over a single long-lived HTTP connection. The client opens the connection with an HTTP request, and the server keeps it open and uses it to send any number of later events. [42] Compared to Long Polling, no new connection is needed per

update, while still running over plain HTTP, which fits most networking setups.

The trade-off is that the proxy and the server must track all connected clients, and the Backend must add routing logic to deliver each event to the right connection. [43]

- **WebSockets**

WebSockets provide two-way communication over a single TCP connection, established by upgrading a standard HTTP connection via the WebSocket Protocol Handshake. [44] Messages can flow in both directions at any moment, which suits applications that need frequent two-way communication. [40]

As with SSE, adding WebSockets requires extra infrastructure and logic for connection and message routing. In addition, WebSocket messages do not follow the HTTP request-response model, so they cannot be cached by standard HTTP caches [45] or CDNs, and they do not benefit from built-in features such as automatic retries.

- **Push API**

The Push API is intended for browser notifications, not for interactive state updates. [46] The server sends messages to a per-domain push endpoint, the browser queues them for a background *service worker*, and the browser may throttle that worker depending on system load. [47, 48] Therefore, latency is higher and less predictable than with direct protocols such as WebSockets, making the Push API unsuitable for time-sensitive updates such as Playback state. [46]

Based on the requirements listed in **Chapter 4**, at least three features would require server-to-client communication:

- **Playback state synchronization.**

Any Playback Device the User acts on has to send the change to the server, whether it is a play, pause, seek, or Song switch, and the Active Playback Device also sends periodic position updates to correct drift. Even at a moderate rate, these events add up, and each one would otherwise need its own HTTP

request. The server then forwards the new Playback state to the User's other Playback Devices to keep them in sync.

- **Real-time typing indicators in Chats and Comment threads.**
The client would send multiple short updates to the server while the User is typing. The server would then broadcast these events to the other participants, so that the current typing activity is displayed in real-time.
- **Live Chat messages and Comments.**
When another User sends a message in an open Chat, or posts a Comment under a Collection that the User is currently viewing, the new content should appear on the screen immediately, without periodic polling from the client.

The initial implementation used SSE. The read-path features in the list above (Live Chat messages, Comments, typing indicators) are mostly server-to-client and would have been covered by a long-lived SSE stream for the receive path with ordinary HTTP requests for the send path. Playback synchronization, however, does not fit this split well. When the User drags the position slider quickly, the primary Playback Device sends a short burst of position updates to the Backend, which then forwards them to the secondary Devices. With SSE for the receive path and one HTTP POST per update on the send path, every update in that burst is a separate HTTP request. Even with a single User this adds extra lag, and with many Users active at the same time it creates significant load on the server.

Additionally, WebSocket support is widely available in modern Python web frameworks, often built in with named groups and per-user broadcast. SSE has less standardized support and typically requires a separate Pub/Sub layer.

For these reasons, WebSockets are used as the primary mechanism for real-time communication. The drawbacks listed above are accepted in exchange for a single two-way channel for everything and lower integration cost with the chosen Backend framework.

These choices define what the Backend and Frontend have to support, and the Backend is discussed next.

5.4 Backend

5.4.1 Language

Python was chosen as the language for the Backend of the application. It is well-suited for web development, since it provides many libraries and built-in interfaces for HTTP APIs, database access, background processing, and other areas used in the implementation. Its syntax also allows fast prototyping, which is helpful while iterating on the data model and the overall app architecture. Performance is good enough for the scale of this project, as the application is not expected to handle very high request rates, and the heaviest task, audio processing, is handed off to external CLI tools (see **Chapter 6**).

5.4.2 Python Framework Comparison

There are currently three widely used frameworks available for Python: FastAPI, Django, and Flask.

- **FastAPI**

As the official website states:

“FastAPI is a modern, fast (high-performance), web framework for building APIs with Python, based on standard Python type hints.” [49]

It is mainly designed to work with Python’s `asyncio`, which makes it much easier to write asynchronous code in comparison to the other two frameworks. In addition, it integrates well with `Pydantic` [50], making user input validation straightforward. Moreover, routing and middleware are easy to configure. [51]

However, FastAPI is relatively new and simple. It can work well for small or highly custom projects, but for standard setups, it lacks many ready-made helpers. For example, the `WebSockets` integration does not handle connections or broadcast messages. [52] Additionally, no predefined templates are available for basic CRUD operations, so they must be implemented manually, which adds extra work.

- **Django**

Django is a “batteries-included” [53] framework in the sense that it has tools for most of the common web development tasks. It provides built-in user session management, easy CSRF and XSS protection setup, built-in file storage abstractions, and integrated handling of media files (images, audio, etc.). Support for REST APIs can be added through `django-rest-framework` [54], which provides generic views, serializers, and routers that make CRUD endpoints quick to implement with little custom code. Real-time functionality can be implemented using `django-channels` [43], an official Django package that handles connections, broadcasts to groups, and integrates with Django’s authentication and session systems. As the documentation states, one of the main advantages of this framework is the time savings it offers. [55]

However, some notable drawbacks include its opinionated design choices, which can make it hard to add custom logic. Furthermore, the size of the resulting codebase can be larger because the framework comes with many modules that may remain unused, while still taking up storage (for example, RSS integration or server-side rendering modules).

Finally, the main disadvantage is its limited support for asynchronous processing [56]: many ORM operations (e.g., transactions), database connection management, and many third-party middlewares/libraries remain synchronous / `async-unsafe`.

- **Flask**

Flask is a lightweight Python web framework that provides only a small core with routing, request/response handling, and templating, while most other functionality is added through extensions. [57] This modular design is suitable mainly for small services or prototypes with a small, clear scope.

However, for typical JSON-based APIs, FastAPI offers similar functionality with fewer dependencies and less setup, while Flask usually requires several additional libraries and manual configuration to do the same thing. At the same time, Flask, like Django, is based on the synchronous WSGI interface [58]. Web-

Socket support is typically added via additional tools, such as SocketIO [59], which couples the Frontend to a specific library instead of the standard WebSocket API. As a result, Flask combines the disadvantages of the other two frameworks: it lacks Django’s integrated ecosystem and does not provide FastAPI’s modern asynchronous features. For this application, it does not bring any clear advantage over Django or FastAPI.

Overall, Django is the most suitable of the three frameworks. For a music streaming platform, its media handling capabilities, WebSocket support via `django-channels`, and CRUD endpoint abstractions of `django-rest-framework` make development faster and shorten the overall implementation time. The limited support for asynchronous code is also less of a problem for this project, since WebSocket handling already requires Django to run under ASGI through `daphne` (which `django-channels` depends on).

Having established the Backend framework, the next step was to choose the database.

5.4.3 Database

The Django framework supports several relational databases: SQLite, PostgreSQL, MySQL/MariaDB, and Oracle. Oracle is excluded due to its commercial license [60]. Otherwise, for a project of this size, any of these options would be fast enough, because the expected number of Users and the amount of stored data are small. Therefore, performance is not the primary factor in choosing a database.

Instead, the choice is based on the available features and how well they are integrated with Django. In this application, two requirements are especially important: full-text search and UUID primary keys (see **Chapter 6** for details). For these requirements, PostgreSQL is the best option.

First, PostgreSQL offers good full-text search support and additional tools for fuzzy matching, such as trigram indexes. [61] As a result, it can be used as the base for full-text search instead of a separate service like Elastic or Sphinx. [62, 63] Django exposes these features through the `django.contrib.postgres` module, which makes it easy to write full-text and trigram-based queries directly from Python code. [64, 65]

Second, PostgreSQL has built-in UUID support via a `uuid` column type. [66] This type stores UUIDs in a compact binary form and allows efficient indexing and comparison.² In many other databases UUIDs must instead be stored as plaintext (e.g., `CHAR(36)`), which uses more space and is slower to index. Django's `UUIDField` maps directly to PostgreSQL's native type, so UUID-based identifiers can be used without extra configuration.

Lastly, with the database chosen, the Frontend application type, language, and framework had to be selected.

5.5 Frontend

5.5.1 Application Type

Modern web applications are typically built either around server-side rendering (SSR), where most of the HTML is generated on the server and sent to the browser to render, or around client-side rendering (CSR), where most HTML is generated in the browser using JavaScript. A single-page application (SPA) is an application that keeps a single HTML document loaded and updates the user interface dynamically on the client, usually relying on CSR, sometimes combined with SSR for the initial page load.

In an SSR setup, frameworks such as Django generate most of the user interface on the server and send complete HTML pages (or partial updates) to the browser, often only requiring small pieces of JavaScript for interaction. [68] In contrast, an SPA typically fetches JSON or other data from the server and, based on it, renders and updates the user interface directly in the browser. [69]

The research for the Frontend began by deciding which of these approaches to use. In this project, the application must maintain a lot of state directly in the browser: the current Playback position, the Playback Queue, the volume level, the Chat state, and so on. These values change frequently and in small steps, sometimes several times per second, and require immediate feedback for the User. Moreover, the user interface must respond to events sent by the server over WebSockets, such as Playback updates or Chat messages. A purely server-rendered

2. If using time-based UUID types, e.g., `UUIDv7` [67]

approach with full page reloads or round-trips for each small change would be unsuitable for this kind of real-time interactive interface and would make state hard to handle across devices. Because of these requirements, the Frontend is implemented as a client-side rendered single-page application (SPA).

5.5.2 Language

Currently, there are many libraries and frameworks in different languages that can be used to write web interfaces. For example, it is possible to create them even in low-level languages, such as C [70] or Rust [71]. However, JavaScript remains the industry standard because it is still the only general-purpose language directly supported by browsers. This means that JavaScript is the only language the browser can run natively, without pre-processing (e.g., compilation to WebAssembly). This simplifies the development, build, and deployment processes, and for this reason the client application is based on JavaScript.

The project uses TypeScript [72], a typed superset of JavaScript that compiles to plain JavaScript. Its main benefit is build-time type checking, which catches many type-related errors before execution.

5.5.3 Framework

Modern Frontend development is mostly based on libraries or frameworks such as React, Vue, or Angular. [73] For this project, React [74] was chosen.

React is a library for building user interfaces from reusable components. Its core only handles rendering, so common needs such as routing, state management, data fetching, and UI components are added through well-established libraries instead of being written from scratch.

React provides a built-in way to manage component state: when a value in that state changes, the parts of the interface that depend on it re-render automatically. This matches the requirements established earlier in this section, since values such as the Playback position, queue, volume, and live Chat messages change frequently and need to appear on screen immediately, without manual DOM updates. React is also

widely adopted [75], so examples and solutions to common problems are easy to find.

5.6 Summary

The platform follows the Model-View-Controller architectural pattern [76] and is organized into three layers:

- **Model**
The Model layer represents and manipulates the application data. In this project, it consists mainly of the Django models with entities such as Users, Songs and Collections, together with the data processing logic, for example audio conversion or data querying.
- **View**
The View layer is what the User interacts with directly. It presents the data from the Model layer and turns User actions into requests that the rest of the system can process. Here, the View is implemented as a single-page application in JavaScript using React. It renders the interface, displays information such as Playback state or Friends' activity, and sends User actions (for example, play/pause commands or Chat messages) to the Controller.
- **Controller**
The Controller layer connects the View with the Model. It receives HTTP and WebSocket requests from the Frontend, applies cookie-based session authentication and access control, and then calls the appropriate Model logic. In this platform, the Controller corresponds to the Django REST API and WebSocket endpoints.

In other words, the Django models and database layer form the Model, the React single-page application acts as the View, and the Django-based REST and WebSocket endpoints act as the Controller that links them.

The next chapter, **Chapter 6**, describes the actual implementation of these parts and how they form the complete music streaming platform.

6 Implementation

This chapter goes over the built application and explains the choices behind each feature. It focuses on what is not clear from the diagrams or the source code. The reader is expected to go through the diagrams and the code first, and then read the sections that explain non-obvious choices. Structure and naming that the diagrams or code already show are not repeated in the text. The high-level system overview is in `text/diagrams/HighLevelSystem.png`, the Backend and Frontend architectures in `text/diagrams/BackendSystem.png` and `text/diagrams/FrontendSystem.png`, and the top-level Frontend component hierarchy in `text/diagrams/FrontendComponentHierarchy.png`.

6.1 Stack

The Backend runs Django 5.2 on Python 3.14, served by the Daphne ASGI server because `django-channels` needs ASGI for WebSocket support [77]. The HTTP API uses Django REST framework with `django-filter` [78], authentication uses `dj-rest-auth` [79] on top of `django-allauth`, and background work runs on Celery. The database is PostgreSQL 17, and Redis [80] is used as both the cache and the data layer for `django-channels`. Dependencies are managed with `uv` [81].

The audio pipeline uses `FFmpeg` [82] for transcoding and `Shaka Packager` [83] for segmentation, both invoked as external CLI tools.

The Frontend is a Vite-built React 19 SPA written in TypeScript, styled with Tailwind and `shadcn/ui`. Queries and mutations are managed via TanStack Query and HTTP calls go through `axios`. Forms use `react-hook-form` with `zod` validation, and Playback uses `Shaka Player` [84] for DASH and HLS streaming.

Deployment is a simple Docker Compose behind a Caddy reverse proxy on a Hetzner VPS, with Mailgun for mail services and Porkbun for the domain.¹

Full dependency lists are located in `src/backend/pyproject.toml` and `src/frontend/musikk/package.json`.

1. The deployed application is available at <https://musikk.stream>.

6.2 Audio Processing and Streaming

This section follows an uploaded audio file through the pipeline that prepares it for Streaming. The Streaming Protocols are covered first, since the rest of the pipeline has to match what they allow.

This section stays at the level of what each stage does and what parameters it uses. The reasoning behind the choices and the per-flag breakdown of every call are in the Audio Processing Pipeline Details appendix. Anything mentioned below but not explained in detail is covered there.

6.2.1 Streaming Protocols

Two protocols are used: MPEG-DASH [85] on devices that support Media Source Extensions (MSE), and HLS [86] on iOS, which does not support MSE that DASH needs. The basic principles behind both were covered in the Segmented HTTP Streaming section. Both protocols expect audio split into short chunks paired with a manifest, produced by the Packaging step described next.

6.2.2 Packaging

Packaging cuts the transcoded audio into short fragmented MP4 segments and generates a DASH manifest (*.mpd) and an HLS master playlist (*.m3u8) that point at them. [87] The segments follow Common Media Application Format (CMAF), so one set of chunks works for both protocols.

FFmpeg can also handle packaging. However, its output had compatibility issues with Shaka Player (the Frontend's Playback library), so a separate packager was chosen.

6.2.3 Transcoding

Transcoding decodes the source audio and re-encodes it into the target codecs and bitrates. Raw audio formats such as WAV are too large for Streaming, for example, a 3-minute stereo 24-bit 44.1 kHz recording is roughly 50 MB. Lossless FLAC and lossy AAC/HE-AACv2 were selected as the output codecs, since HLS only allows a small set of

audio codecs. [88] The AAC encoder used is `libfdk_aac`, chosen over the other AAC encoders bundled with FFmpeg for its quality and licensing terms.

The output configuration was chosen based on existing research: [89, 90, 91, 92, 88, 93]

- **AAC:** 96, 160, and 320 kbps
- **HE-AACv2:** 24 kbps
- **FLAC:** lossless

Multiple bitrates support adaptive streaming. The AAC tiers use Constant Bitrate (CBR), so the Frontend's Adaptive Bitrate logic has a stable per-tier bandwidth to pick from.

Transcoding is done with one FFmpeg process per output tier in parallel, through a thread pool sized to the number of tiers. Threads are used rather than processes because FFmpeg itself is multi-threaded inside each subprocess, so the Python side only waits on the subprocess to finish.

Transcoding needs a clean input with reasonable parameters. The Audio Preprocessing step described next ensures that each upload is valid and within the proper limits, and also captures the metadata of the result.

6.2.4 Audio Preprocessing

Audio Preprocessing has two parts. Before Transcoding, the upload is validated and normalized. After Transcoding, the loudness is measured.

Input Validation

The upload passes through a chain of checks. The file-size and MIME checks run on the raw bytes. The rest read values from FFprobe: codec allow-list, duration, sample rate, and channel count. The exact limits and the allow-lists are listed in the Audio Processing Pipeline Details appendix.

Input Metadata Extraction

FFprobe reads the source codec, sample rate, channel count, bit depth, duration, and bitrate. These values are reused by the input validation above. They also narrow which outputs the Transcoding step produces: FLAC is skipped for lossy sources, and AAC tiers whose target bitrate is above the source bitrate are dropped.

Normalization

Three normalization filters run as a part of the Transcoding command:

- **Channel normalization.** Sources with three or more channels are downmixed to stereo. Mono and stereo inputs are kept as-is.
- **Sample rate normalization.** Each source is resampled to the base of its rate family (44.1 kHz or 48 kHz). Files below 44.1 kHz are kept as-is. Non-family rates default to 44.1 kHz.
- **Bit depth conversion.** All audio is converted to signed 16-bit samples, since `libfdk_aac` does not accept higher precision.

Loudness Measurement

After Transcoding, FFmpeg's `ebur128` filter produces an integrated value in Loudness Units relative to Full Scale (LUFS) and a true peak in dBTP. The measurement runs on the FLAC output if it was produced, otherwise on the first AAC output. Both values are stored on the Song and used by the Frontend player to apply a consistent Playback gain.

Once Transcoding and Packaging finish, the original upload and the intermediate transcoded MP4s are removed, since they are not needed for Playback. If a pipeline step fails, the previously executed steps are rolled back in reverse order, clearing the per-run temporary directory and removing any output generated during the processing.

When this pipeline is started, and how its status reaches the Frontend, is covered in the Audio Upload section.

6.2.5 Frontend Playback

As mentioned in the Stack section, Shaka Player handles DASH and HLS playback on the Frontend. The audio element it controls is owned by the `AudioPlayerController`, described in the Playback and Device Handling section.

This concludes the audio pipeline. The next section covers the WebSocket layer.

6.3 WebSocket Communication

This section covers the WebSocket setup of the application. The Backend code is under `src/backend/musikk/websockets`, the Frontend code under `src/frontend/musikk/src/ws`. The end-to-end connection lifecycle is in `text/diagrams/WebsocketLifecycle.png`, and the class layout in `text/diagrams/WebsocketsClassDiagram.png`.

Two WebSocket message-related terms are used throughout the text: *Events* are messages sent from the server to the client and *Actions* are messages sent from the client to the server.

6.3.1 Backend Handling

All real-time features are multiplexed over a single connection at `/ws/user`. With one connection per feature, extra message ordering logic would need to be implemented. For example, a Playback Device switch and a Playback activation could arrive in any order, and it would not be clear which one to process first. In addition, each extra connection adds memory and file-descriptor overhead on the server, can run into the browser's connection limit [94], and needs its own reconnect handling on the Frontend.

As discussed in Server-Client Communication, WebSocket support is added via `django-channels`. The `BaseConsumer` class manages connections and is instantiated once per connection.

Two kinds of broadcast groups are used. The User's own group is used for private messages such as the Playback Device list, Notifications, and Liked Songs changes. All of the User's tabs join this group, so a private broadcast reaches every tab. Named topic groups are

joined by several Users and used for shared messages: Chat messages, Collection Comments, and typing indicators.

A Playback event can include an excluded device id, in which case the consumer skips delivery to that device. This is useful because usually the device that sent the update already has the new state and does not need to receive its own event back.

6.3.2 Frontend Handling

The Frontend opens a single connection to `/ws/user`. Authentication is implicit: the browser sends the session cookie along with the upgrade request. The WS-handling code is split into two classes, `WSCliient` and `WebSocketManager`.

The client also implements reconnect logic, so a tab that is put to the background or a brief network outage does not require a page refresh or manual actions from the User. Reconnects use exponential backoff with a 30 s ceiling and random jitter. The jitter prevents many tabs from reconnecting at the same time after an outage.

`WSCliient` wraps the native `WebSocket` interface. Outgoing actions are queued and flushed when the connection is open. An action issued while the connection is down is therefore not lost, and the order of queued actions is preserved on flush. The exception is the periodic Playback position sync, which is sent only if the connection is open. Otherwise stale seek actions could be replayed on reconnect, even if the Playback position has already moved and stabilized.

Named topic subscriptions are handled via the `WSCliient` as well. Subscriptions are reference-counted: only the first topic subscriber runs a subscribe action, and only the last unsubsubscriber runs the unsubscribe action. Both actions also bypass the queue. On reconnect, the on-open handler reads the refcount map and sends a subscribe for every topic in it. If subscribe also used the queue, the server would get the same subscribe twice. If unsubscribe used the queue, a subscribe and an unsubscribe sent while the connection was down would still reach the server on reconnect, even though the Frontend no longer needs the topic.

`WebSocketManager` owns the single `WSCliient` via a refcount. `connect` returns the existing client and increments the counter. `disconnect` closes the connection only when the counter reaches zero. With-

out the refcount, one subscriber unmounting (including React's Strict-Mode double-mount in development) would tear down the connection shared with the others, drop their topic subscriptions, and force a fresh handshake plus re-subscription on the next mount.

Feature code does not touch either class directly. Instead it uses hooks (e.g., `useChatWsEvents`) that subscribe to a topic and register listeners for the matching events.

The WebSocket connection is also used for Frontend cache invalidation. The tab that changes the data invalidates the affected React Query key on mutation success. Every other open tab invalidates the same key when the server event for new data arrives. The sequence diagram is in `text/diagrams/WsCacheInvalidation.png`.

6.4 User Management

This section covers how Users are modelled, how the Frontend renders them, and the API that connects the two. The Backend code is under `src/backend/musikk/users`, the Frontend code under `src/frontend/musikk/src/features/user`. The class diagram is in `text/diagrams/UsersClassDiagram.png`, and the Frontend component hierarchy in `text/diagrams/FrontendUser.png`.

6.4.1 User Data Model

A single `BaseUser` model covers every account, with a role discriminator field that is either "Base" or "Artist". An Artist is not a separate subclass, only a role value, since there are currently no Artist-only fields (a separate database table is not needed).

The `display_name` is not unique and defaults to a random string, so a new account always has a usable name before the User picks one.

User-to-User relations are modelled by a single `UserFollow` table. Friendship is not a separate model. A Friend is inferred from a mutual follow, two `UserFollow` rows in opposite directions. The `BaseUser.friends` property computes that join directly in the ORM, so the Friend set is never stored.

6.4.2 Backend

The endpoints are listed in `src/backend/musikk/users/api/v1/urls.py`.

The `/users/me` endpoint splits the serializer by HTTP method. GET returns the public fields plus the User's own email. PATCH exposes only `display_name`, `bio`, and `avatar`. The split exists so that the `role`, `email`, and `is_artist` flag cannot be written through the same endpoint as profile edits, even if extra fields are sent in the payload. The public retrieve endpoint called for another User omits the email, so an email address only ever reaches the owner of the account.

A follow does more than insert a row. The POST handler is idempotent through `get_or_create`, so a double-click does not create a duplicate notification. A successful follow also creates a `FollowerNotification` that appears in the target's Notifications. Unfollow is a DELETE on the same path. It broadcasts `USER_UNFOLLOWED` to both groups and removes the matching `FollowerNotification`, so the target's Notification window does not keep an invalid entry.

Three list endpoints under `/users/<uuid>/` (`friends`, `followers`, `followed`) are protected by the `IsSelfOrFriend` permission, so the connection lists are visible only to the owner or to one of their Friends. The social relations of a User are not considered public.

6.4.3 Frontend

`ProfilePage` covers both the own-profile and other-User views with one component, switching between an "Edit Profile" dialog and a "Follow/Unfollow" button based on whose UUID is in the route. The page layout is otherwise identical between the two views, so a split into two components would mainly duplicate code and need to be kept in sync.

6.5 Account Creation and Authentication

This section covers how a new account is created, how the email address is verified, and how the resulting Session is established and maintained. The Backend code is under `src/backend/musikk/users` and the Frontend code under `src/frontend/musikk/src/features/`

auth. The end-to-end flow across sign-up, email confirmation, login, and session expiry is in `text/diagrams/AuthFlow.png`. The Frontend component hierarchy is in `text/diagrams/FrontendAuth.png`.

6.5.1 Backend

The Backend does not implement its own authentication stack, since session handling, password validation, and email verification are security-sensitive and easy to get wrong. [95] Instead authentication is built using `django-allauth` [96] and REST endpoints on top of `allauth` are provided by `dj-rest-auth` [79] (located in `/api/v1/auth/`).

`allauth` is configured to identify Users by email, and the username field is disabled. Sign-up therefore takes only email, password, and a password confirmation. `BaseRegisterSerializer` adds a write-only `is_artist` boolean and sets the new User's role to "Artist" when the flag is true, so the role choice is set during sign-up rather than requiring a separate upgrade step.

A custom `BaseAccountAdapter` class overrides the default authentication flow. `clean_email` rejects an email that already exists case-insensitively, since the stock `allauth` check is case-sensitive and would let `user@x.com` and `User@X.com` register as separate accounts, which RFC 5321 advises against. [97]

Email verification is set to mandatory, i.e., a new account exists in the database immediately after registration, but login is blocked until the link in the verification email has been opened.

6.5.2 Frontend

Two axios clients are configured in `src/frontend/musikk/src/api/axiosConf.ts`. The split exists because the authenticated client defines a response interceptor that triggers a global logout on 401, which would loop on the login request. Both clients share a request interceptor that copies the `csrftoken` cookie into the `X-CSRFToken` header on unsafe methods.

The 401 handler on the authenticated client uses a shared `authRef.logout` reference rather than calling `logout` directly. The reference is set inside an effect in `AuthProvider` and cleared on unmount. This keeps the axios module free of React imports and avoids a full-

page redirect, which would remount the app and lose any in-progress operations before the logout can finish.

The next section covers the main objects Users interact with: Songs and Collections.

6.6 Songs and Collections

This section covers the Song/Collection models and their Author relations. The Backend code is located under `src/backend/musikk/streaming`. The Frontend code is split across the `songs/` and `collections/` subdirectories under `src/frontend/musikk/src/features/`. See `text/diagrams/StreamingClassDiagram.png` for the class diagram.

The audio pipeline fields on Song and the Album upload flow are covered in the Audio Upload section. Playback Queue design is covered in Song Queue and Playback logic in Playback and Device Handling.

6.6.1 Data Model

BaseSong holds the fields that make each Song playable and identifiable: title, description, image, draft flag, and the paths to the streaming manifests.

The app has only one Song container, the Collection class. It has four kinds, Playlist, Album, History, and Liked Songs, which are discriminated by a string field, the same pattern as the role field on BaseUser. This makes it possible to reuse one set of retrieve, list, and ordering operations, and only one Collection page component on the Frontend for all four kinds. All four kinds share the same fields and the same operations, so the single-model approach works well here. The same shape did not hold for Notifications, which are split into separate models in the Notifications section.

CollectionSong pairs a BaseSong with a Collection at an integer position. It also serves as the public identifier of a Song for most of the API endpoints. For example, `/songs/<uuid>` resolves to a CollectionSong, not a BaseSong. The reason is that most actions on a Song target its slot in a specific Collection, not the Song in the abstract.

Examples include “Remove from this Playlist”, “Jump to the Album”, and rendering the play button state. A URL that accepts a `BaseSong` would have to either include the Collection UUID in every call as well, or do a lookup to find the row.

Authorship is stored in two join tables, `SongCredit` and `CollectionCredit`. `SongCredit` restricts the author to Users with the Artist role, while `CollectionCredit` accepts any User, since Playlists can be created by regular Users as well.

6.6.2 Backend

The endpoints are listed in `src/backend/musikk/streaming/api/v1/urls.py`.

The Collection manager exposes per-kind query helpers, so the type filter is not repeated in each view that lists Collections.

Liking a Song uses a separate endpoint from the generic Playlist add, even though both create a `CollectionSong` row. This keeps Frontend cache handling simple. The like endpoint emits a separate `LIKED_SONGS_CHANGED` event that invalidates one query holding the set of liked Song UUIDs, so every like button on the page re-renders from a single signal. The earlier design routed likes through the generic Playlist endpoint, but it left the like state as a per-row `is_liked` flag, refreshed only by the generic `COLLECTION_CHANGED` event. As a result, the same Song shown in an Album view and the Playback Queue could stay out of sync.

6.6.3 Frontend

Two components, `SongCard` and `CollectionCard`, are reused throughout the application. Both are written to be embeddable, so any other component can include them without extra setup.

6.7 Audio Upload/Album Creation

This section covers the design choices behind the Audio Upload. The end-to-end sequence is in `text/diagrams/AudioUploadFlow.png`, and the Frontend component hierarchy in `text/diagrams/`

FrontendAudioUpload . png. The Backend view Collection-SongCreateView is at `src/backend/musikk/streaming/api/v1/views/collections.py`. The Celery task `convert_audio` is at `src/backend/musikk/streaming/audio/tasks.py`. The Frontend code is under `src/frontend/musikk/src/features/album-upload`.

Transcoding a long file can take minutes. The view therefore validates the file, writes it to a temporary path, creates the Song row with `draft=true`, enqueues a Celery task, and returns 202 Accepted with the new Song UUID without waiting for the audio processing to finish. Otherwise the HTTP request would either time out behind the reverse proxy or block the Frontend from showing progress.

The Frontend generates an *operation id* per upload and sends it in the request. The Celery task echoes the id in every WebSocket event it emits, so the Frontend can match events to uploads when several run in parallel.

The view passes only the temporary path to the task, not the file bytes. This is because Celery serializes task arguments through Redis and keeps every queued message in RAM, so passing raw bytes would let a few concurrent uploads fill the broker's memory.

A Song that is still processing has no audio data yet. It is therefore created with `draft=true`, and the flag is cleared only after the Celery task writes the final audio fields. Public list and retrieve endpoints filter on `draft=false`, so a half-built Song is not accessible on the Frontend. The Album has the same flag, and the Artist publishes the Album with a separate request once every Song is processed.

Data cleanup is split from the Celery task. The task removes the files it created on failure, but leaves the row in place, because the Artist may retry a failed Song or come back later. A separate management command, `prune_draft_albums`, removes Album rows that have stayed in draft past a configurable window, together with their Songs and segments, and is meant to be run from cron.

This concludes the upload path. The next section covers the Song Queue, the structure the User selects Playback Items from.

6.8 Song Queue

This section covers how the User can enqueue playable items in the application. The model code is under `src/backend/musikk/streaming/models/song_queue.py`, the Frontend component hierarchy is in `text/diagrams/FrontendSongQueue.png`, and the Backend sequence for adding a Collection and advancing is in `text/diagrams/SongQueue.png`.

6.8.1 Data Structure

The Queue supports several frequent operations: insert at a random index, reorder, remove an item, pop from the front, pop several items up to a certain one, bulk append multiple items, and read the upcoming items in order. The first draft used a doubly linked list in SQL. However, this did not work well for most of these operations. Every insert, reorder, or remove cascaded pointer updates, and reading items in order needed a linear traversal rather than a single range scan. A decimal index approach was chosen instead, based on online discussions on this topic [98, 99]. The current model stores each `QueueItem` with a `Decimal` position and inserts a new item at the midpoint between its neighbours. A single insertion is one DB write, and reads are a simple range query.

The trade-off is that the position value can run out of precision when the User keeps inserting at the same spot. The position column is `Decimal(24, 12)`, so it has 12 fractional digits. This is enough for about 30 midpoint inserts between the same two neighbours before no number is left to put between them. However, in practice, the User rarely edits the Queue this deeply, so this limit is fine for now. Appends use a separate `next_item_position` counter instead. The counter grows by a fixed gap, so appended items never land on the same position and never use up the fractional digits.

6.8.2 Queue and PlaybackContext

The Queue itself is split into two classes. `SongQueue` holds items the User added explicitly, ordered by position. `PlaybackContext` tracks the Collection that is currently playing through a cursor into `Col-`

lectionSong.position and a list of skipped indices. On advance, the Queue is consumed first, then the Context. If no items are left, Playback stops. The split lets the User add items on top of a playing Collection, work through them, and then return to where the Collection was.

6.8.3 QueueSource and Lazy Filling

Appending a large Collection does not add all items immediately. A QueueSource stores either a single Song or a Collection with a cursor, and converts Sources into Items in batches when the number of Items in the Queue table drops below 30. Each fill creates up to 20 items. This keeps the Queue table small even when the User queues an entire Album with hundreds of tracks, and it lets the Frontend show a stable window of upcoming items without paging through thousands of rows.

With the Queue covered, the next section turns to how Playback state is kept consistent across the User's tabs and Devices.

6.9 Playback and Device Handling

This section covers how Playback state is kept in sync across a User's tabs and Devices. The Backend code is under `src/backend/musikk/streaming`, with WS action handlers in `src/backend/musikk/streaming/ws/action_handlers/playback.py`, and the per-User state stored in Redis via `PlaybackManager`. The Frontend code is under `src/frontend/musikk/src/features/player` and `src/frontend/musikk/src/features/playback`. The Backend models are in `text/diagrams/PlaybackClassDiagram.png`. The runtime managers, controllers, and broadcasters are in `text/diagrams/StreamingWsClassDiagram.png`. The Frontend component hierarchy is in `text/diagrams/FrontendPlayback.png`, and the multi-tab sync sequence in `text/diagrams/PlaybackSync.png`.

Playback is server-authoritative. The Backend keeps one `PlaybackState` object per User (covering all User connections) in Redis. Every WS action from any client updates that state, and the server then broadcasts the updated state to every connected User tab.

An earlier design let each tab keep its own state and used the server only to pass updates to the other tabs. That design was dropped because the server could not order the events. Suppose tab A sends a seek to 30 s and tab B sends a seek to 90 s 50 ms later. Network jitter could swap the order in which the two broadcasts reach a third tab, so its position would end at 30 s even though 90 s was the newer action. The server had no way to pick a winner. The server-authoritative model fixes this because every state change goes through a single ordered path and gets a fresh version number, described in the next subsection.

6.9.1 Shared State and Version Counter

PlaybackState holds two pieces: `ActivePlayback`, which stores the actual Playback values, and a monotonically increasing version number. `ActivePlayback` is a separate object because it only exists while a Song is loaded (when nothing is playing, the field is `None`). The version number is kept on `PlaybackState` itself, so it survives across pauses and Song changes.

The version increments on every “real” state change (seek, pause, etc., not clock sync). Every event sent to clients includes the current version. A tab keeps the version of its last applied event and drops any broadcast whose version is older than that, which protects against out-of-order arrivals like the one described above. The Backend itself does not use the version to drop incoming actions. Instead, every mutating action from a tab includes the Song UUID the tab thinks is current, and the Backend drops the action if that UUID does not match the one in `ActivePlayback`. A mismatch means the tab is acting on a Song the User has since moved past.

The play instance UUID is regenerated when a new Song becomes the active Playback item. It disambiguates Queue entries that point at the same Song. Queue entries are `CollectionSongs`, and the User can put the same `CollectionSong` into the Queue more than once in a row. Without the UUID, resuming after a skip to the second occurrence would inherit the position saved for the first.

6.9.2 Position Estimation on Passive Tabs

Exactly one Device is the active Playback Device at any time. Only the active tab loads the current Song into its HTML audio element. If every tab loaded the Song, the User would hear several streams at once.

The audio element stays mounted on every tab, even when empty. The Web Audio API pipeline (covered later in this section) binds to this element. Keeping it mounted on every tab lets the same pipeline survive tab switches without rebuilding.

The active tab reads the current Song position directly from its audio element. Passive tabs do not have a Song loaded, but they still need to show the position slider. They therefore estimate the position from the server's last known value.

Only the active tab reports the position to the server. It reads the `currentTime` attribute of its audio element and sends that value. The server stores the pair `last_known_song_pos_ms` (the reported position) and `last_known_at_server_ms` (the server clock when it arrived). Any client estimates the current position as `last_known_song_pos_ms` plus the time that has passed since `last_known_at_server_ms`. Clients do not have direct access to the server clock. Each broadcast includes the server's current timestamp, and the client compares it with its own local clock at receive time to learn how far the two clocks are apart. It then adds that gap to its local clock whenever it needs to estimate what the server clock reads now. The limits of this simple clock-gap method are discussed in Playback Synchronization in the improvements chapter.

The active tab updates the pair when its audio position diverges from its own estimate by more than 500 ms, and on a 20 s heartbeat otherwise. The heartbeat is needed because the drift check itself depends on the active tab's estimate of the server clock, which gets less accurate without fresh broadcasts.

6.9.3 Device Activation

The active Device id is tracked on the server alongside the Device list, in a separate Redis key from `PlaybackState`. The Backend never fills it automatically. A User can have several tabs and Devices open at

once, and the Backend cannot tell which one the User currently uses. Another reason to avoid an auto-pick: the User click that sets the active Device also counts as the browser gesture needed to start audio on that tab (see Browser autoplay handling below).

When no Device is active, every tab is passive. The first action that needs Playback sets the active Device. Play actions (including skip to the next or previous Song) send `device.set_active` before the play action on the Frontend. The seek action takes a different path on the Frontend, so the Backend sets the active Device when it receives a seek with no Device active.

A change of active Device is a `device.set_active` WS action (initiated by the Device dropdown) followed by a broadcast of the new Device list. The Backend pauses Playback on the transition, so the newly active tab does not try to resume audio without a User gesture on that tab, which the browser would block.

6.9.4 Browser Autoplay Handling

Browser autoplay policies [100] add two constraints. First, browsers only let the page create or resume an `AudioContext` (the Web Audio API starting point used by the pipeline in the next subsection) while the browser is still handling a User click or key press. The Frontend listens for the first `pointerdown` or `keydown` anywhere on the page and builds the pipeline inside that handler, so the first User action counts as the gesture. Second, `audio.play()` itself is gesture-gated. If the User had another tab playing, then closed it and resumes from a tab they have not interacted with yet, `audio.play()` rejects with `NotAllowedError`. The Frontend catches this and retries with the audio element muted, which browsers always allow, and the UI shows an Unmute control. The Unmute click is the gesture that lets the Frontend resume the `AudioContext` and switch the audio element back to unmuted.

6.9.5 Frontend Playback Control

The Frontend Player is a class, `AudioPlayerController`, not a React hook. An earlier version of the Frontend used a hook (named `use-Player`). In that version, each WebSocket handler held a “frozen copy” of the React Playback state from the moment it was attached, and was

never re-attached, so later events kept reading the old values. The class form removes this problem because it owns the Playback state directly as a field. React components subscribe to the class through `useSyncExternalStore` and re-render on changes, while the WS handlers read the same field directly and always see the current value.

The Playback needs per-Song loudness normalization, which cannot be done via the HTML audio element. The Frontend therefore routes audio through a Web Audio API graph between the audio element and the speakers. An `AudioContext` is the starting point for any Web Audio API work, and all graph nodes are created through it. The first stage applies the normalization from the stored LUFS measurement of the Song. A compressor at the end of the chain catches peaks that the loudness boost can push past the clipping point. The other two stages are the User's volume slider, and short volume fades around play, pause, and seek, since abrupt cuts make audible clicks.

With Playback covered, the next section turns to Publications: Posts, Collection Comments, Chat messages, and Replies.

6.10 Publications

This section is about Posts, Collection Comments, Chat messages, and Replies. The Backend code is under `src/backend/musikk/social`. The Frontend code is under `src/frontend/musikk/src/features/publications`. The class diagram is in `text/diagrams/SocialClassDiagram.png`. The Frontend component hierarchy is in `text/diagrams/FrontendPublications.png`, and the Chat connection lifecycle in `text/diagrams/ChatLifecycle.png`.

All four kinds use the same Django model, `Publication`. The model stores the author, the text, an optional parent `Publication` for Replies, and a `created_for` field that points to the object the `Publication` is written for. This object is a `User` for Posts, a `Collection` for Collection Comments, and a `Chat` for Chat messages. An optional attachment field can point to a `Song` or a `Collection`. The shared model was chosen because every kind has the same basic shape: a `User` writes some text, the text can be a Reply to another `Publication`, and it can have a `Song` or a `Collection` attached. This shared model has some downsides that are discussed in Backend Refactors.

At first, the API was built the same way. One endpoint took the `created_for` type and id, and chose the right logic inside. This became hard once the rules for each kind started to differ. Collection Comments are only allowed on public, non-draft Albums or Playlists. Posts are only allowed on the User's own Feed. Chat messages are only allowed between Chat members and need two extra models, Chat and ChatMember, which do not fit into the shared row. Per-kind logic for permissions, WebSocket events, and Notifications also differs, so every request to the shared endpoint had to start with a `created_for_type` switch.

The API was therefore split into one endpoint per kind: `/feed/<user_uuid>/posts`, `/collections/<uuid>/comments`, and `/users/<user_uuid>/chats/<chat_uuid>/messages`. Each view keeps the rules for its kind in one place, so this branching is no longer needed. A shared mixin still holds the common create and list code, and per-kind views override what differs between kinds. Replies work the same way for all four kinds, because a Reply has the same shape regardless of its root Publication. They are created through the same endpoint as the root, with the parent field set. They are read through one shared endpoint, `/publications/<uuid>/children`. The database keeps one table for all kinds, and the API uses the names that the Frontend already expects.

6.11 Search

This section covers full-text Search across Songs, Albums, Playlists, Users, and Artists. The Backend code is under `src/backend/musikk/recommendations`, the Frontend code under `src/frontend/musikk/src/features/search`.

The Frontend talks to the Backend through one endpoint, `GET/search?q=...`. The response has five groups: Songs, Albums, Playlists, Users, and Artists. Each bucket has up to ten matches, sorted by similarity. Items with a score at or below 0.3 are dropped. Song matches are returned as their Collection Song row from a public Album, so the Frontend can reuse the same Song card it uses elsewhere.

As stated in **Chapter 5**, the Backend uses PostgreSQL with trigram matching, available to Django through `django.contrib.postgres.-`

search. `TrigramSimilarity` was chosen because it scores the whole field against the query, which fits the short labels searched here: Song titles, Collection titles, and User display names. The word-level trigram variants in the same module are aimed at longer text, and do not work well with shorter values. The module also exposes PostgreSQL full-text search, which is a separate mechanism that matches whole words and their grammatical forms rather than character trigrams. However, full-text search does not handle typos by default, and this UI needs that.

6.12 Notifications

This section shows how Notifications are modelled, emitted, and delivered. The Backend code is under `src/backend/musikk/notifications`. The Frontend code is under `src/frontend/musikk/src/features/notifications`. The class diagram is in `text/diagrams/NotificationsClassDiagram.png`, and the Frontend component hierarchy in `text/diagrams/FrontendNotifications.png`.

There are three Notification kinds: a Reply to a Publication the User authored, a new Follower, and a new Chat message. The first design used one shared Notification base with three concrete subclasses through Django's multi-table inheritance. Each row held an `is_read` boolean. This worked well at the start, since every kind had the same two operations, list and mark read, and one query covered all of them. However, the kinds soon gained more fields of their own, and the shared base no longer fit well. A Reply points to two Publications, a Follower to two Users, and a Chat message to a Chat together with the message and the receiver. The columns almost do not overlap. As a result, a shared row was mostly nulls, and every read needed an extra join into one of the concrete tables. In contrast to Publications, which still use one shared model with mostly-null rows, Notifications have been split into three separate models.

Each kind is its own `BaseModel` subclass with its own `Serializer` and custom `Manager`. In comparison to other parts of the application, managers here are used to emit a `notifications.changed` WS event instead of the API views. This coupling between the data layer and the WebSocket layer is discussed further in Backend Refactors.

Read state was also made simpler. The per-row `is_read` boolean was replaced by one `NotificationProfile.read_at` timestamp per User. The profile is created on User registration through a `post_save` signal handler. The Serializer computes `is_read` for each row by comparing `date_added` against that timestamp. As a result, “mark all as read” becomes one field update.

What still stays shared is the REST API seen by the Frontend. One `GET /notifications` endpoint returns a response with three buckets. A `PATCH` on the same path updates `read_at`. A `DELETE /notifications/<uuid>` checks the three tables in turn to find the matching row. This shape still suits the Frontend, which fetches all kinds together for the badge and the list view. However, the differences between kinds are no longer hidden behind a common base.

6.13 Security

This section goes over the OWASP guidelines [101] the application addresses and the gaps that remain.

6.13.1 Transport and Origin

Caddy [102] terminates TLS in production, and Django upgrades plain HTTP to HTTPS. Frontend, REST API, and the WebSocket endpoint share the same origin (covered in the Deployment section), so CORS headers and `django-cors-headers` are not needed.

6.13.2 Authentication and Session Management

The Backend follows the OWASP Authentication [103] and Session Management [104] cheat sheets through Django defaults, `django-allauth`, and DRF throttling. The only recommended control not implemented is account lockout after a fixed number of failed logins. The per-endpoint rate limit on login is the only protection.

6.13.3 XSS and CSRF

XSS is mostly mitigated, because the Frontend relies on React's default escaping [74], and no part of the code bypasses it (e.g., via `dangerouslySetInnerHTML`).

Django's `CsrfViewMiddleware` provides token-based protection per the OWASP CSRF Prevention cheat sheet [105]. Same-origin routing already blocks cross-origin forgery in the browser, so the middleware is kept as defense in depth.

6.13.4 Input Validation

Audio uploads follow the OWASP File Upload cheat sheet [106], with the checks described in the Audio Processing and Streaming section. All database access goes through the Django ORM, so SQL injection is not a surface. The only gap left is that image uploads are accepted but not re-encoded.

6.13.5 WebSocket Security

The WebSocket setup (authentication on upgrade, Origin pinning, per-topic subscribe validation) is covered in the WebSocket Communication section and matches the OWASP WebSocket security guidance [107]. The remaining gap is that action handlers have no per-connection rate limit.

6.13.6 HTTP Response Headers

The OWASP Secure Headers project [108] recommends a small set of response headers. The current reverse proxy setup sets most of these, with three omitted. `Strict-Transport-Security` should be set only when the domain is stable, since a browser that caches the policy locks the site into HTTPS for the full max-age. `Content-Security-Policy` requires an SPA-aware policy setup. `Permissions-Policy` is low priority, since the application does not use the camera, microphone, or geolocation APIs.

6.14 Deployment

This section covers the production deployment setup. The Docker Compose file is at `docker-compose-prod.yml`. The Backend and Frontend Dockerfiles are at `src/backend/Dockerfile` and `src/frontend/Dockerfile`. The Caddy configuration is at `src/frontend/Caddyfile.prod`.

The Backend, the Postgres database, the Redis instance, and the Celery worker run as four containers under Docker Compose on a Hetzner-hosted server. Caddy is the reverse proxy in front of them. It was chosen mainly for its simple syntax and automatic DNS certificate setup. Email is sent through the Mailgun service via `django-anymail`, and the domain is managed at Porkbun. FFmpeg is built from source inside the container with `libfdk_aac` enabled, since the default FFmpeg builds do not include it [109]. The resulting image is not published in a public registry, since the FFmpeg with `libfdk_aac` binary cannot be redistributed due to licensing issues. [110]

With the application completely ready, user testing results and possible future improvements are discussed in the next chapter.

7 User Testing and Further Improvements

7.1 Further Improvements

This section lists known gaps and possible next steps for the app.

7.1.1 Out-of-Scope Items

On the Frontend, several features are missing. There is no mobile layout and no offline Playback, and the interface is English-only. Accessibility is also only partial. For example, some clickable elements use `div` instead of `button`, and the custom controls have no Accessible Rich Internet Applications (ARIA) roles or labels. There is also no Digital Rights Management (DRM) mechanism.

In addition, a few Backend features have no Frontend counterpart. The password reset and password change endpoints exist under `/api/v1/auth/`, but the SPA has no UI for them. Similarly, the Queue view does not support drag-to-reorder, even though the Backend has a reorder endpoint.

Finally, production troubleshooting relies on reading the Daphne and Celery logs over SSH. There is no error reporter, no metrics pipeline, and no structured log format.

7.1.2 Search and Recommendations

Search ranks results only by how close the strings match, and does not use any signal from the User's listening history. A mix of trigram and full-text search, with a small re-ranker based on what the User has played, could improve the results.

In addition, there is no Recommendations engine and the Backend endpoint that adds random Songs to the Queue currently returns HTTP 501.

7.1.3 Playback Synchronization

The clock calculation method from Playback and Device Handling only takes the difference between the client and server clocks at receive time. As a result, on slow networks the estimate is off by about the

one-way packet trip time. A better method would record both the send time and the receive time of each broadcast, use half the round-trip as the offset, and average several samples (this is the approach NTP uses). [111]

7.1.4 Backend Refactors

The three Notification managers send `notifications.changed` inside `Manager.create`. As a result, the data layer is coupled to the WebSocket layer. The broadcast call should move into the view or a small service object, the same way other parts of the app do it.

A similar problem appears in the Publications section, which uses one Django model for Posts, Collection Comments, Chat messages, and Replies. The data is similar, but the behavior for each kind is very different. For example, Collection Comments are only allowed on public non-draft Collections, and Posts are only allowed on the User's own Feed. Chat messages, in turn, need Chat and ChatMember support. The API is already split into one endpoint per kind. Therefore, the next step is to split the model in the same way Notifications were split.

7.1.5 Frontend Cache Strategy

Almost every WebSocket event handler on the Frontend calls `queryClient.invalidateQueries`, even when the event payload already has the new state. As a result, every change forces a refetch and shows a short loading state. Where the payload is enough, the handler should call `queryClient.setQueryData` instead, and keep invalidation only as a fallback. This would remove the extra round-trip.

7.1.6 Test Coverage

Backend test coverage is not high. The Notifications app has no tests at all, the audio conversion tests only cover lossless source inputs, and the endpoints are not tested thoroughly. Moreover, the Frontend has no automated tests at all. Component tests with React Testing Library and some smoke tests for the main flows would be a good next step.

8 Conclusion

The goal of this thesis was to design and implement a music streaming platform with built-in features for social interaction around music. The motivation came from two sources. The survey in **Chapter 2** showed that listening to and sharing music is a common social activity, and that around 60% of respondents would use extra social features if a streaming platform offered them. The review of existing platforms in **Chapter 3** then showed that current services cover only a subset of these features, which forces Users to switch between several apps. Based on this, requirements were defined with the MoSCoW method (**Chapter 4**), the main technologies were picked through a comparison of the available options (**Chapter 5**), and the application itself was built and described in **Chapter 6**.

The main outcome of this work is the deployed application, which gathers in one place the social features that are otherwise split across the services reviewed in **Chapter 3**. These are added on top of a standard music-streaming core: audio upload with multi-bitrate Transcoding, MPEG-DASH and HLS Streaming, a Playback Queue, and Search across Songs, Collections, Users, and Artists. The Playback state is server-authoritative and synchronized across a User's open tabs and Devices over a single WebSocket connection. The application covers all the "Must Have" and "Should Have" requirements from **Chapter 4**.

Compared to the platforms reviewed in **Chapter 3**, no single existing service combines all of these features. Spotify Jam offers shared Playback and Chat, SoundCloud has Comments under tracks, VK Music has shared posts, and last.fm has profile feeds, but each service covers only a part of the set.

8.1 Closing Remarks

The outcome of this thesis can be considered successful. The application meets the requirements defined during the planning phase, the architecture is documented, and the code is open to extension. The project covers a wide set of topics, from audio Streaming and real-time communication to social features and full-stack deployment.

The thesis has greatly improved the author's programming and system design abilities, as well as research and information validation skills.

A Audio Processing Pipeline Details

These notes expand on the *Audio Processing and Streaming* section of **Chapter 6**. The main chapter names every stage briefly and lists the chosen parameters. The sections below give the fuller reasoning for each choice and a per-flag breakdown of every CLI command.

A.1 Streaming Protocols

DASH was considered first, because it works better in varying network conditions [112] and is not limited to a small set of audio codecs. [30] However, DASH support in audio players is usually built on top of browser Media Source Extensions [113], which iOS does not support. [114] Since iOS makes up about 15% of all operating systems worldwide [115], HLS support had to be added as well.

A.2 Packaging

Two notes on CMAF packaging. First, packager tools rarely state in their output that the segments follow CMAF. This can be checked by running, for example, the `ffprobe` CLI tool on a packaged segment. Second, the resulting output is split into one initialization segment (`*_init.mp4`), which stores audio metadata and decoder setup, and a sequence of media segments (`*.m4s`), which store the audio data in short time intervals.

In addition, fMP4 segments are required for streaming FLAC encoded (lossless) audio via HLS. [116]

A.3 Processing Tools

Three CLI tools are used in the processing pipeline: FFmpeg for Transcoding and audio analysis, FFprobe for reading audio metadata, and Shaka Packager for the Packaging step.

FFmpeg supports all of the selected codecs, can be run from a script, and converts audio at high speed because it is written in C and

uses multiple threads. The pipeline uses it both as the transcoder and as the host for the filter chain applied during Audio Preprocessing.

FFprobe comes with the FFmpeg project and reads metadata from audio files. The Audio Preprocessing pipeline uses it to read the source codec, sample rate, channel count, bit depth, duration, and bitrate of every upload before Transcoding.

A.4 Transcoding

A.4.1 AAC Encoder Choice

FFmpeg comes with several AAC encoder implementations, only one of which is used here:

- `aac_at` is the best codec in terms of quality. [117, 118, 119] However, it was not chosen because its license only allows use on Apple-branded devices. [120, 109]
- `aac` is FFmpeg's own AAC implementation. It was also rejected, since the quality it gives is worse than the other two codecs listed here (this is even stated in the official documentation). [117, 118, 119, 109]
- `libfdk_aac` is the second best codec in terms of quality and the one that was chosen. It also has license limits: any binaries, Docker images, and other distribution methods that include it cannot be made public or freely sent to other devices. However, it can be used free of charge in closed-source projects. [110, 121, 109]

For FLAC, FFmpeg provides a single `flac` encoder, which is used directly. [122]

A.4.2 Bitrate Strategy: CBR Versus VBR

Bitrate describes how much audio data is stored or sent per second. It is usually given in kilobits per second (kbps) and is the main way to balance audio quality against file size.

Two encoding strategies exist: Constant Bitrate (CBR) [123] and Variable Bitrate (VBR). [124] CBR keeps the bitrate fixed across the

entire audio file, so each representation has a known, stable bandwidth. VBR changes the bitrate based on content complexity, which gives better quality at a smaller size compared to CBR, but the per-segment size varies within a single representation. [125]

A multi-CBR setup was chosen because the Frontend's Adaptive Bitrate logic needs a stable per-tier bandwidth to pick from. The same effect could in principle be reached with constrained VBR tiers, but the fixed per-tier bitrate is simpler for the project scope.

A.4.3 Generation Loss

Transcoding a lossy source into AAC causes generation loss [126]. Every lossy pass throws away some audio, so re-encoding what was already lossy makes the loss worse. The pipeline still does the conversion because DASH and HLS need every representation in the same codec. With FLAC and AAC chosen as the output codecs, lossy uploads have to be re-encoded into AAC anyway.

A.5 Audio Preprocessing

A.5.1 Input Validation Limits

The upload passes through a chain of validation checks. The file-size and MIME checks run on the raw bytes. The remaining checks read values from the file with FFprobe. Raw audio storage grows linearly with duration, sample rate, channel count, and bit depth, so the limits together cap the worst-case storage and Transcoding time of a single upload:

- **MIME type.** The first 2048 bytes of the file are inspected with `python-magic` [127], and the type must start with `audio/`. The subtype must also belong to an allow-list of subtypes that map to the supported codecs, defined in the `ALLOWED_FILE_TYPES` variable in `streaming/audio/config.py`. This step catches malformed or non-audio uploads before any audio tool is started.
- **Codec allow-list.** A container can hold different codecs (for example, an `.mp4` file can hold AAC or ALAC), so the codec name FFprobe reports must also belong to `ALLOWED_CODECS`

in `streaming/audio/config.py`. The list only covers popular codecs.

- **File size: 2 GB.** Roughly a 2-hour 24-bit, 48 kHz stereo WAV. An arbitrary number, but real music files usually stay well below this, so the limit mostly guards against accidental or malicious oversize uploads.
- **Duration: up to 7200 s (2 h).** Covers DJ sets and long mixes while keeping the upload and Transcoding time reasonable.
- **Sample rate: 1 Hz to 384 kHz.** The upper bound matches the highest rate in the supported sample rate families, described in the Sample rate normalization step below.
- **Channel count: 1 to 8.** Covers mono through 7.1 surround, which is the practical upper bound for source audio.

A.5.2 FFprobe Invocation

```
ffprobe -v error -select_streams a \
-show_entries stream=codec_name,sample_rate,channels,bits |
    ↪ _per_raw_sample,duration,bit_rate \
-show_entries format=duration,bit_rate \
-of json input.wav
```

The flags are:

- `-v error` hides informational and warning output and keeps only actual errors.
- `-select_streams a` limits probing to audio streams.
- `-show_entries stream=...` returns only the per-stream fields the wrapper uses.
- `-show_entries format=...` adds a fallback for duration and `bit_rate`, since some containers (notably WAV) only report them at the format level.
- `-of json` produces structured output that the wrapper parses.

A.5.3 Channel Normalization

The normalization step reduces every track to mono or stereo. Music releases are mostly mastered in stereo, and the chosen Streaming Protocols and the loudness measurement step below are also defined for mono or stereo input. Surround formats such as Dolby Atmos [128] need a different container, extra information in the manifest, and per-platform decoder support, none of which fit the project scope. [129]

Mono inputs stay mono, since upmixing to stereo would not add information, only file size. Stereo inputs stay stereo. Sources with three or more channels are downmixed to stereo by adding `aresample=out_chlayout=stereo` to the FFmpeg filter chain.

A.5.4 Sample Rate Normalization

Human hearing tops out around 20 kHz. The Nyquist-Shannon theorem [130] says a sample rate must be at least twice the highest frequency in the signal to represent it without aliasing. A 44.1 kHz rate therefore covers the audible band with a small margin, and source rates above that give no extra audio quality, only larger files. [131]

Music masters use one of two standard rates, 44.1 kHz or 48 kHz [132], or an integer multiple of one of them. The step groups each source into its family and resamples down to the family base. An integer-ratio resample is fast to compute and stays close to the original signal. Files below 44.1 kHz are left as they are, since upsampling adds no information. Non-family rates default to 44.1 kHz. Resampling is done by adding `aresample=<rate>` to the FFmpeg filter chain.

A.5.5 Bit Depth Conversion

All audio is converted to signed 16-bit samples through `aformat=sample_fmts=s16`. The conversion is always applied, because `libfdk_aac`, the encoder used for the AAC tiers, does not accept higher-precision sample formats. FLAC accepts higher precision, but using the same filter chain for every tier keeps the Transcoding command the same across tiers.

A.5.6 Loudness Theory

Loudness normalization keeps the perceived volume steady across tracks [133]. Human hearing is not flat: equal-loudness contours described by ISO 226 [134] show that perceived loudness depends on both amplitude and frequency, so two files with the same digital peak can sound very different. Without normalization, a playlist that mixes tracks from different masters would have the listener constantly reaching for the volume knob.

ITU-R BS.1770 [135] is the standard algorithm for measuring loudness. It produces an integrated value in Loudness Units relative to Full Scale (LUFS), using a frequency weighting (K-weighting) made to match how humans hear and a channel weighting model. EBU R128 [136] turns the algorithm into a normalization recommendation, with a broadcast target of -23 LUFS. Streaming services use higher targets because listening conditions differ from broadcast, for example Spotify at -14 LUFS [137] and Apple Music at -16 LUFS. [138]

Two other units appear here. Decibels Full Scale (dBFS) measure sample amplitude relative to the largest value the format can store, so 0 dBFS is the largest sample and recorded peaks are usually negative. True peak in dBFS (dBTP) measures the peak of the reconstructed analog waveform after digital-to-analog conversion, not the peak of the stored samples. The difference matters: a sequence of samples that are all below 0 dBFS can still reconstruct to over 0 dBFS, producing inter-sample peaks that clip on Playback. True-peak measurement uses an oversampled signal to catch this.

A.5.7 Why Loudness Is Measured After Transcoding

FFmpeg's `ebur128` filter with `peak=true` produces both an integrated LUFS value and a true peak in dBTP. The measurement runs after Transcoding, on the FLAC output if it was produced, otherwise on the first AAC output. FLAC is preferred when available because it keeps the normalized signal bit-perfectly. The measured values therefore describe the normalization output without being changed by the lossy AAC encoder.

The measurement happens after Transcoding rather than on the source file for two reasons. First, the listener hears the audio after

processing. Channel downmix changes overall loudness, since each channel adds to the perceived sum. Sample-rate conversion can also shift true peaks slightly. The stored values therefore have to describe what actually plays. Second, the ebur128 summary reports the peak as a per-channel list on multichannel inputs. For example, a six-channel source produces a summary line such as Peak: -1.1 -2.3 -1.5 -2.1 -3.0 -1.8 dBFS, which is unclear as a single per-track value. After downmix to stereo the list has at most two entries, and the wrapper records the maximum of the two.

A.6 Pipeline Invocations

A.6.1 FFmpeg Transcoding Flags

The FFmpeg command for the AAC 160 kbps tier on a stereo 44.1 kHz source is:

```
ffmpeg -i input.wav \
  -vn -sn -dn \
  -af "aresample=out_chlayout=stereo,aresample=44100,aforma_
    ↪ t=sample_fmts=s16" \
  -c:a libfdk_aac \
  -b:a 160k \
  -movflags +faststart \
  libfdk_aac-160-<uuid>.mp4
```

The flags are:

- `-i input.wav` selects the input media file.
- `-vn -sn -dn` strip any video, subtitle, and data streams, so only the audio stream is processed.
- `-af "..."` applies the audio filter chain for normalization (channel downmix, sample-rate normalization, bit-depth conversion).
- `-c:a libfdk_aac` selects the audio encoder. The other encoders used by the pipeline are `flac` for the lossless tier and `libfdk_aac` with the extra `-profile aac_he_v2` flag for the HE-AACv2 24 kbps tier.

- `-b:a 160k` sets the target audio bitrate. This flag is left out for the FLAC tier, since it is lossless.
- `-movflags +faststart` moves the MP4 metadata (the `moov` atom [139]) to the start of the file, so Playback can begin before the whole file is downloaded.
- The last positional argument is the output file path. The wrapper uses the pattern `<encoder>-<bitrate>-<uuid>.mp4` for the lossy tiers and `flac-static-<uuid>.mp4` for the lossless tier.

A.6.2 Shaka Packager Flags

The Shaka Packager command for two of the AAC tiers is:

```
packager \
'input=/tmp/in/libfdk_aac-96-<uuid>.mp4,stream=audio,init_s|
  → egment=/tmp/out/audio_0_init.mp4,segment_template=/|
  → tmp/out/audio_0_${Number$.m4s,playlist_name=audio_0.|
  → m3u8' \
'input=/tmp/in/libfdk_aac-160-<uuid>.mp4,stream=audio,init_|
  → segment=/tmp/out/audio_1_init.mp4,segment_template=|
  → /tmp/out/audio_1_${Number$.m4s,playlist_name=audio_1|
  → .m3u8' \
--segment_duration 3 \
--generate_static_live_mpd \
--mpd_output=/tmp/out/manifest.mpd \
--hls_master_playlist_output=/tmp/out/master.m3u8
```

The Backend builds this command as a Python list and runs it through subprocess, without a shell, so the `${Number$}` token is passed to the packager as-is. The example above wraps each per-input argument in single quotes so that bash does not expand the token.

The flags are:

- `packager` is the Shaka Packager binary.
- Each `input=...` argument defines one representation. The keys are:
 - `input=<path>` is the local path to the transcoded audio file.

- stream=audio selects only the audio stream from that file.
 - init_segment=<path> sets the output path for the CMAF/fMP4 initialization segment (*_init.mp4).
 - segment_template=<path> sets the template for media segment file names. \$Number\$ is a placeholder that the packager replaces with the segment index.
 - playlist_name=<name> sets the file name of the per-representation HLS variant playlist, for example audio_0.m3u8.
- -segment_duration 3 sets the target segment length in seconds. The value of 3s was chosen as a balance between how fast the player can switch bitrates and the cost of each HTTP request.
 - -generate_static_live_mpd switches the DASH manifest type from dynamic (the default) to static, which is what audio-on-demand clients expect and which lets the player seek through the track.
 - -mpd_output=<path> sets the output path for the DASH manifest (manifest.mpd).
 - -hls_master_playlist_output=<path> sets the output path for the HLS master playlist (master.m3u8).

A.6.3 ebur128 Loudness Measurement Flags

The FFmpeg command for the loudness pass, run on the FLAC output if it was produced or otherwise on the first AAC output, is:

```
ffmpeg -i flac-static-<uuid>.mp4 \
  -filter:a ebur128=peak=true \
  -f null -
```

The flags are:

- -i <path> selects the transcoded audio file the wrapper picked for measurement.

A. AUDIO PROCESSING PIPELINE DETAILS

- `-filter:a ebur128=peak=true` runs the ebur128 audio filter. `peak=true` turns on true-peak measurement (dBTP) alongside the integrated LUFS value, so the wrapper gets both numbers from a single pass.
- `-f null` - discards the audio output, since only the summary the filter prints on stderr is needed. The trailing `-` tells FFmpeg to write the (discarded) output to stdout.

The wrapper parses the printed summary, extracts the I: (LUFS) and Peak: (dBTP) values, and stores them on the Song row.

Bibliography

- [1] Peter J. Rentfrow. *The Role of Music in Everyday Life: Current Directions in the Social Psychology of Music*. URL: <https://compass.onlinelibrary.wiley.com/doi/abs/10.1111/j.1751-9004.2012.00434.x>.
- [2] *Sharing Music: Social and Communal Aspects of Concert-Going*. URL: <https://ojs.meccsa.org.uk/index.php/netknow/article/view/428/250>.
- [3] *Music Streaming Market Decade Insights*. URL: <https://www.businessofapps.com/data/music-streaming-market/>.
- [4] *Google Forms*. URL: <https://docs.google.com/forms/u/0/>.
- [5] *Spotify Popularity*. URL: <https://simplebeen.com/music-streaming-statistics>.
- [6] *Spotify Will Suspend Its Services in Russia in Light of Free Speech Crackdown*. URL: <https://techcrunch.com/2022/03/25/spotify-will-suspend-its-services-in-russia-in-light-of-free-speech-crackdown/>.
- [7] *Yandex Music*. URL: <https://music.yandex.com/>.
- [8] *VK Music*. URL: <https://music.vk.com/>.
- [9] *Zvuk*. URL: <https://zvuk.com/>.
- [10] *Audio Usage Statistics 2024*. URL: https://www.ifpi.org/wp-content/uploads/2023/12/IFPI-Engaging-With-Music-2023_full-report.pdf.
- [11] *Spotify Jam*. URL: <https://support.spotify.com/cz/article/jam/>.
- [12] *Spotify Friend Activity*. URL: <https://support.spotify.com/cz/article/friend-activity/>.
- [13] *Spotify Party Announcement*. URL: <https://newsroom.spotify.com/2024-09-12/fans-artists-connections-features/>.
- [14] *Spotify Party*. URL: <https://support.spotify.com/us/article/listening-party/>.
- [15] *Spotify Chat*. URL: <https://newsroom.spotify.com/2025-08-26/introducing-messages-a-new-way-to-share-what-you-love-on-spotify-with-friends-and-family/>.
- [16] *SoundCloud Comments*. URL: <https://help.soundcloud.com/hc/en-us/articles/115003566008-Commenting-basics>.

BIBLIOGRAPHY

- [17] *SoundCloud Reactions*. URL: <https://help.soundcloud.com/hc/en-us/articles/31039497560731-Reactions>.
- [18] *Soundcloud Reposts*. URL: <https://help.soundcloud.com/hc/en-us/articles/115003567488-Reposting-tracks-or-playlists>.
- [19] *Fans on Bandcamp*. URL: <https://bandcamp.com/fans>.
- [20] *Rate Your Music*. URL: <https://rateyourmusic.com/discover>.
- [21] *Last.fm*. URL: <https://en.wikipedia.org/wiki/Last.fm>.
- [22] *Instagram Audio*. URL: https://help.instagram.com/664508917489819?helpref=faq_content.
- [23] *Music in This Video: YouTube Help*. URL: <https://support.google.com/youtube/answer/7680188>.
- [24] *TikTok and Billboard Partner to Launch the TikTok Billboard Top 50 Chart*. URL: <https://newsroom.tiktok.com/en-us/tiktok-billboard-top-50-chart>.
- [25] Dai Clegg and Richard Barker. *Case Method Fast-Track: A Rad Approach*. URL: <https://dl.acm.org/doi/10.5555/561543>.
- [26] *Multipurpose Internet Mail Extensions Part Two: Media Types*. 'octet-stream'. URL: <https://www.rfc-editor.org/rfc/rfc2046#section-4.5.1>.
- [27] *Nginx pseudo-streaming implementation*. URL: https://nginx.org/en/docs/http/nginx_http_mp4_module.html.
- [28] *HTTP Range Requests*. URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Range_requests.
- [29] Issing Jochen Färber Nikolaus Döhla Stefan. *Adaptive progressive download based on the MPEG-4 file format*. URL: <https://link.springer.com/article/10.1631/jzus.2006.AS0106>.
- [30] *The MPEG-DASH Standard for Multimedia Streaming Over the Internet*. URL: <https://ieeexplore.ieee.org/abstract/document/6077864>.
- [31] *AWS CDN Caching*. URL: <https://aws.amazon.com/caching/>.
- [32] Beng Chin Ooi Quang Hieu Vu Mihai Lupu. *Peer-to-Peer Computing: Principles and Applications*. URL: https://books.google.tt/books?id=kd8_AAAQBAJ&printsec=frontcover&source=gbs_vpt_read#v=onepage&q&f=false.
- [33] *Basic Authentication*. URL: <https://datatracker.ietf.org/doc/html/rfc7617>.

-
- [34] *Overview of TLS termination and end to end TLS with Application Gateway*. URL: <https://learn.microsoft.com/en-us/azure/application-gateway/ssl-overview>.
 - [35] *Session Authentication*. URL: <https://docs.djangoproject.com/en/5.2/topics/http/sessions/>.
 - [36] *Cross Site Scripting Prevention Cheat Sheet*. URL: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html.
 - [37] *Using HTTP Cookies*. URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies>.
 - [38] Michael B. Jones, John Bradley, and Nat Sakimura. *RFC 7519: JSON Web Token (JWT)*. URL: <https://www.rfc-editor.org/rfc/rfc7519>.
 - [39] *OAuth2*. URL: <https://datatracker.ietf.org/doc/html/rfc6749>.
 - [40] *Comparison of Server to Client Communication Methods*. URL: <https://www.diva-portal.org/smash/get/diva2:1133465/FULLTEXT01.pdf?ref=hackernoon.com>.
 - [41] *Known Issues and Best Practices for the Use of Long Polling and Streaming in Bidirectional HTTP*. URL: <https://datatracker.ietf.org/doc/html/rfc6202>.
 - [42] *Server Sent Events*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events.
 - [43] *Django Channels*. URL: <https://channels.readthedocs.io/en/latest/>.
 - [44] *WebSockets*. URL: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API.
 - [45] *Http Caching*. URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Caching>.
 - [46] *Push API*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Push_API.
 - [47] *Generic Event Delivery Using HTTP Push*. URL: <https://datatracker.ietf.org/doc/html/rfc8030>.
 - [48] *Service Worker API*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API.
 - [49] *FastAPI*. URL: <https://fastapi.tiangolo.com/>.
 - [50] *Pydantic*. URL: <https://docs.pydantic.dev/latest/>.

-
- [51] *FastAPI Features*. URL: <https://fastapi.tiangolo.com/features>.
 - [52] *FastAPI Websockets*. URL: <https://fastapi.tiangolo.com/advanced/websockets/>.
 - [53] *Django*. URL: <https://www.djangoproject.com/>.
 - [54] *DRF*. URL: <https://www.django-rest-framework.org/>.
 - [55] *Django Overview*. URL: <https://www.djangoproject.com/start/overview/>.
 - [56] *Django Async Support*. URL: <https://docs.djangoproject.com/en/5.2/topics/async/>.
 - [57] *Flask*. URL: <https://flask.palletsprojects.com/en/stable/>.
 - [58] Phillip J. Eby. *PEP 3333: Python Web Server Gateway Interface v1.0.1*. URL: <https://peps.python.org/pep-3333/>.
 - [59] *SocketIO*. URL: <https://socket.io/>.
 - [60] *Oracle Price List*. URL: <https://www.oracle.com/a/ocom/docs/corporate/pricing/technology-price-list-070617.pdf>.
 - [61] *PostgreSQL Full Text Search*. URL: <https://www.postgresql.org/docs/current/textsearch.html>.
 - [62] *Elasticsearch*. URL: <https://www.elastic.co/elasticsearch>.
 - [63] *Sphinx Search Engine*. URL: sphinxsearch.com.
 - [64] *PostgreSQL*. URL: <https://www.postgresql.org/about/>.
 - [65] *Django PostgreSQL Support*. URL: <https://docs.djangoproject.com/en/5.2/ref/contrib/postgres/>.
 - [66] *Postgres UUID Type*. URL: <https://www.postgresql.org/docs/current/datatype-uuid.html>.
 - [67] *UUIDv7*. URL: <https://www.thenile.dev/blog/uuidv7>.
 - [68] *Django Templates*. URL: <https://docs.djangoproject.com/en/5.2/ref/templates/language/>.
 - [69] *Single Page Application*. URL: <https://developer.mozilla.org/en-US/docs/Glossary/SPA>.
 - [70] *Clay Library*. URL: <https://github.com/nicbarker/clay>.
 - [71] *Yew Framework*. URL: <https://github.com/yewstack/yew>.
 - [72] *TypeScript: JavaScript with Syntax for Types*. URL: <https://www.typescriptlang.org/>.
 - [73] *Framework Usage*. URL: <https://2024.stateofjs.com/en-US/libraries/>.
 - [74] *React*. URL: <https://react.dev/>.

-
- [75] *StackOverflow React*. URL: <https://stackoverflow.com/questions/tagged/reactjs?tab=Newest>.
 - [76] *MVC*. URL: <https://developer.mozilla.org/en-US/docs/Glossary/MVC>.
 - [77] *Daphne GitHub Page*. URL: <https://github.com/django/daphne>.
 - [78] *django-filter*. URL: <https://django-filter.readthedocs.io/en/stable/>.
 - [79] *dj-rest-auth*. URL: <https://pypi.org/project/dj-rest-auth/>.
 - [80] *Redis*. URL: <https://redis.io/>.
 - [81] *uv*. URL: <https://docs.astral.sh/uv/>.
 - [82] *FFMPEG*. URL: <https://ffmpeg.org/>.
 - [83] *Shaka Packager*. URL: <https://shaka-project.github.io/shaka-packager/html/>.
 - [84] *Shaka Player*. URL: <https://github.com/shaka-project/shaka-player>.
 - [85] *MPEG-DASH*. URL: <https://www.mpeg.org/standards/MPEG-DASH/>.
 - [86] *HLS*. URL: <https://developer.apple.com/streaming/>.
 - [87] *Packaging explanation*. URL: <https://shaka-project.github.io/shaka-packager/html/documentation.html>.
 - [88] *HTTP Live Streaming (HLS) authoring specification for Apple devices. Bitrate recommendations, section 2.25*. URL: <https://developer.apple.com/documentation/http-live-streaming/hls-authoring-specification-for-apple-devices#Bitrate-recommendations-for-Ambisonics-are-as-follows>.
 - [89] *EBU Evaluations of Multichannel Audio Codecs*. URL: <https://tech.ebu.ch/docs/tech/tech3324.pdf>.
 - [90] *Perceived Audio Quality for Streaming Stereo Music*. URL: <https://dl.acm.org/doi/pdf/10.1145/2647868.2655025>.
 - [91] *BACHELOR THESIS Perceived Audio Quality in Ogg Vorbis and AAC in Compressed Popular Music in Bit Rates between 96 kbit/s, 128 kbit/s and 192 kbit/s*. URL: <https://www.diva-portal.org/smash/get/diva2:1030609/FULLTEXT02.pdf>.
 - [92] *Spotify Audio Quality*. URL: <https://support.spotify.com/cz/article/audio-quality/>.

-
- [93] *Web audio codec guide*. URL: https://developer.mozilla.org/en-US/docs/Web/Media/Guides/Formats/Audio_codecs#aac_advanced_audio_coding.
 - [94] *Domain Sharding: MDN Web Docs Glossary*. URL: https://developer.mozilla.org/en-US/docs/Glossary/Domain_sharding.
 - [95] Kevin Fu et al. *Dos and Don'ts of Client Authentication on the Web*. URL: <https://users.umi.acs.umd.edu/~tudor/courses/ENEE757/Fall14/papers/Fu01.pdf>.
 - [96] *Welcome to django-allauth!* URL: <https://docs.allauth.org/en/latest/>.
 - [97] John C. Klensin. *RFC 5321: Simple Mail Transfer Protocol*. URL: <https://www.rfc-editor.org/rfc/rfc5321#section-2.4>.
 - [98] *Linked List in SQL 1*. URL: <https://stackoverflow.com/questions/65205/linked-list-in-sql>.
 - [99] *Linked List in SQL 2*. URL: <https://dba.stackexchange.com/questions/46238/linked-list-in-sql-and-trees>.
 - [100] Mounir Lamouri. *Autoplay Policy in Chrome*. URL: <https://developer.chrome.com/blog/autoplay>.
 - [101] *OWASP Cheat Sheet Series*. URL: <https://cheatsheetseries.owasp.org/>.
 - [102] *Caddy*. URL: <https://caddyserver.com/>.
 - [103] *Authentication Cheat Sheet*. URL: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html.
 - [104] *Session Management Cheat Sheet*. URL: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html.
 - [105] *Cross-Site Request Forgery Prevention Cheat Sheet*. URL: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html.
 - [106] *File Upload Cheat Sheet*. URL: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html.
 - [107] *WebSocket Security Cheat Sheet*. URL: https://cheatsheetseries.owasp.org/cheatsheets/WebSocket_Security_Cheat_Sheet.html.
 - [108] *OWASP Secure Headers Project*. URL: <https://owasp.org/www-project-secure-headers/>.

-
- [109] *ffmpeg libfdkaac*. URL: <https://trac.ffmpeg.org/wiki/Encode/AAC>.
 - [110] *Via Licensing*. URL: <https://via-la.com/licensing-programs/aac/#faqs>.
 - [111] David L. Mills et al. *RFC 5905: Network Time Protocol Version 4: Protocol and Algorithms Specification*. URL: <https://www.rfc-editor.org/rfc/rfc5905>.
 - [112] *Comparative Analysis of MPEG-DASH and HLS Protocols: Performance, Adaptation, and Future Directions in Adaptive Streaming*. URL: <https://link.springer.com/article/10.1007/s40031-025-01244-x#Sec4>.
 - [113] *MSE*. URL: <https://www.w3.org/TR/media-source-2/>.
 - [114] *MSE Availability*. URL: <https://caniuse.com/mediasource>.
 - [115] *Operating System Market Share Worldwide*. URL: <https://gs.statcounter.com/os-market-share>.
 - [116] *HTTP Live Streaming (HLS) authoring specification for Apple devices. Audio*. URL: <https://developer.apple.com/documentation/http-live-streaming/hls-authoring-specification-for-apple-devices#Audio>.
 - [117] *Personal Listening Test of AAC-LC and xHE-AAC at 96kbps and 128kbps*. URL: <https://hydrogenaudio.org/index.php?topic=119861.0>.
 - [118] *Personal Blind Listening Test of AAC at 128 kbps (six encoders & 105 samples)*. URL: <https://hydrogenaudio.org/index.php?topic=120062.0>.
 - [119] *Qaac VS FDK vs FGh vs FAAC ? Which one is better now in 2021?* URL: <https://hydrogenaudio.org/index.php/topic,120759.0.html>.
 - [120] *SOFTWARE LICENSE AGREEMENT FOR macOS Tahoe 26 (section "J. Other Use Restrictions")*. URL: <https://www.apple.com/legal/sla/docs/macOSTahoe.pdf>.
 - [121] *Fraunhofer FDK AAC Codec Library*. URL: <https://spdx.org/licenses/FDK-AAC.html>.
 - [122] *ffmpeg flac codec*. URL: <https://ffmpeg.org/ffmpeg-codecs.html#toc-flac-2>.
 - [123] *CBR*. URL: <https://learn.microsoft.com/en-us/windows/win32/wmformat/constant-bit-rate--cbr--encoding>.

- [124] VBR. URL: https://wiki.hydrogenaudio.org/index.php?title=Variable_Bitrate.
- [125] CBR vs VBR. URL: <https://ieeexplore.ieee.org/abstract/document/6774590>.
- [126] Generation Loss. URL: https://en.wikipedia.org/wiki/Generation_loss.
- [127] *python-magic Package*. URL: <https://pypi.org/project/python-magic/>.
- [128] *Dolby Atmos: Official Site*. URL: <https://www.dolby.com/technologies/dolby-atmos/>.
- [129] *Recommendation ITU-R BS.775: Multichannel Stereophonic Sound System with and without Accompanying Picture*. URL: <https://www.itu.int/rec/R-REC-BS.775/en>.
- [130] Claude E. Shannon. *Communication in the Presence of Noise*. URL: https://fab.cba.mit.edu/classes/S62.12/docs/Shannon_noise.pdf.
- [131] Dan Lavry. *The Optimal Sample Rate for Quality Audio*. URL: https://www.lavryengineering.com/pdfs/lavry-white-paper-the_optimal_sample_rate_for_quality_audio.pdf.
- [132] AES5-2018: *AES Recommended Practice for Professional Digital Audio. Preferred Sampling Frequencies for Applications Employing Pulse-Code Modulation*. URL: <https://www.aes.org/publications/standards/search.cfm?docID=14>.
- [133] *Audio Normalization*. URL: https://en.wikipedia.org/wiki/Audio_normalization.
- [134] ISO 226:2023 *Acoustics: Normal Equal-Loudness-Level Contours*. URL: <https://www.iso.org/standard/83117.html>.
- [135] *Recommendation ITU-R BS.1770: Algorithms to Measure Audio Programme Loudness and True-Peak Audio Level*. URL: <https://www.itu.int/rec/R-REC-BS.1770/en>.
- [136] *EBU R 128: Loudness Normalisation and Permitted Maximum Level of Audio Signals*. URL: <https://tech.ebu.ch/publications/r128>.
- [137] *Loudness Normalization on Spotify*. URL: <https://support.spotify.com/us/artists/article/loudness-normalization/>.

BIBLIOGRAPHY

- [138] Ian Stewart. *Apple Choose -16 LUFS Loudness Level For Apple Music: Here's Why*. URL: <https://www.production-expert.com/production-expert-1/apple-choose-16lufs-loudness-level-for-apple-music-heres-why>.
- [139] *Movie atom ('moov')*. URL: https://developer.apple.com/documentation/quicktime-file-format/movie_atom.